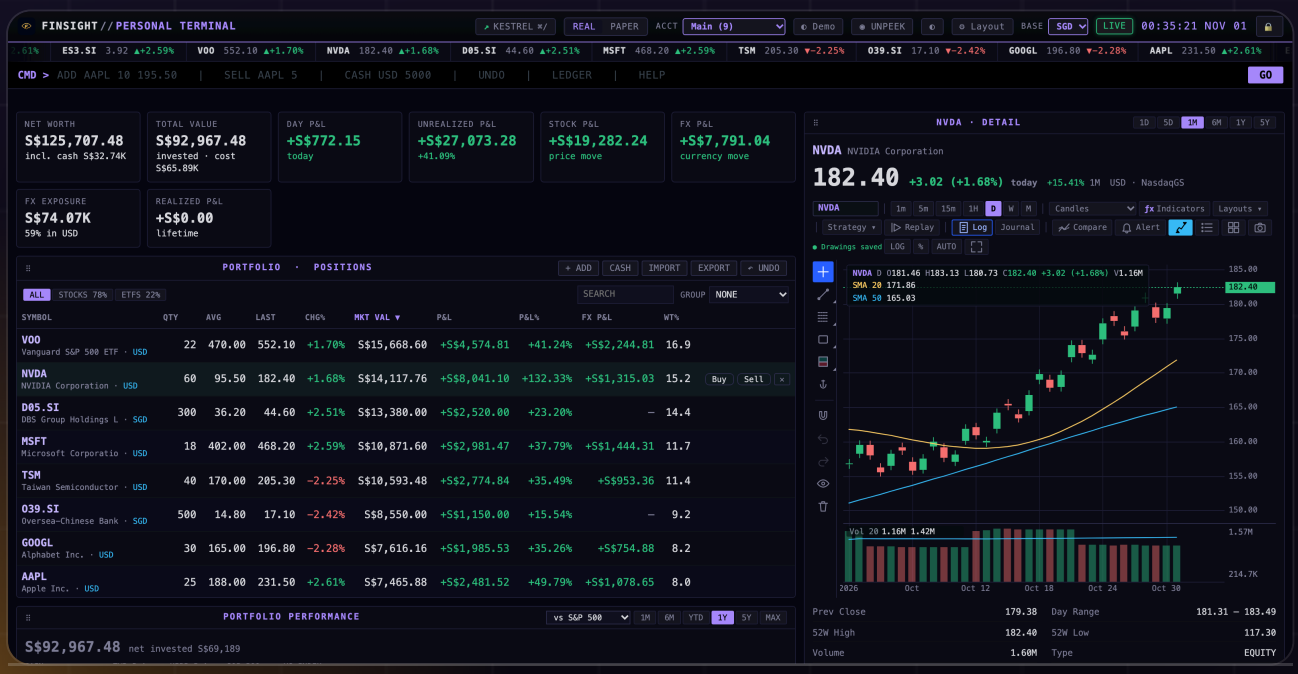
FINSIGHT // TERMINAL

MASTER PRODUCT & ENGINEERING DOCUMENT

FinSight Terminal & Kestrel Agent

Product · Architecture · Agent · Security · Quality

Product, requirements, architecture, agent design, security, concurrency, quality and operations in one place.



VERSION	DATE	OWNER	STATUS
1.0	1 November 2026	tanueihorng	Living document · reflects main at aa373b7

Contents

Document control

1 Executive summary

2 Problem and motivation

3 Product overview

4 Stakeholders and personas

5 User stories

6 User flows

7 Functional specification

8 Non-functional requirements

9 System architecture

10 Software specification

11 Technology stack

12 Kestrel agent design

13 Investment council design

14 Paper trading design

15 Security

16 Concurrency and consistency

17 Reliability and error handling

18 Software engineering practice and SDLC

19 Testing strategy

20 UAT plan for computer-use testing

21 Deployment and operations

22 Repository structure

23 Feature status and roadmap

24 Risks, assumptions and constraints

25 Engineering review against 24 domains

Appendix A — Glossary

Appendix B — Terminal commands

Appendix C — Kestrel commands

Appendix D — Traceability matrix

Appendix E — References

Document control

Field	Value
Document	FinSight Terminal — Master Product & Engineering Document
Version	1.0 (1 November 2026)
Owner	tanueihorng (product owner, sole user, final decision-maker)
Code baseline	`main` at `aa373b7` — Merge paper trading, trade desk, Kestrel limits (FEAT-012)
Audience	New contributors, coding agents, reviewers, demo audiences and UAT testers (including Codex computer use)
Companion files	`docs/guide/finsight-guide.html` (interactive demo guide), `agent/docs/SPEC.md`, `docs/architecture/SYSTEM.md`, `docs/BACKLOG.md`

How to read this document

- **Sections 1–4** explain the problem, the product and who it is for. Read these first.
- **Sections 5–8** are the requirements: user stories, user flows, the functional specification and non-functional requirements.
- **Sections 9–15** are the engineering design: architecture, software specification, tech stack, the Kestrel agent, the council and paper trading.
- **Sections 16–18** cover security, concurrency and reliability.
- **Sections 19–22** cover how the project is built and shipped: SDLC, testing and the UAT catalogue, operations and repository layout.
- **Sections 23–24 and the appendices** cover status, risks, glossary and traceability.
- **Section 25** reviews FinSight against 24 software-engineering domains, with evidence, gaps and an improvement backlog.

> **Naming**
> **Kestrel** is FinSight's built-in AI research agent (sometimes misheard as "Cashier" or "Cursor" in voice notes). **Council** is Kestrel's multi-specialist review mode. **Paper trading** is the Alpaca-backed fake-money book.

> **Status words**
> This document distinguishes **implemented** (code exists), **verified** (tests/evidence recorded), **merged** (on `main`) and **deployed** (running in the owner's personal install). Figures shown in examples come from the synthetic demo portfolio, not from real holdings.

1. Executive summary

FinSight is a free, Bloomberg-style portfolio terminal that runs entirely on the owner's computer. It tracks a multi-currency, multi-account portfolio with live public market data, and it answers a question most tools hide: **how much of my return came from the stock, and how much from the currency?**

On top of the terminal sits ↗ **Kestrel**, a local AI research agent that can see what the user sees — holdings, cash, P&L, events, strategies and the chart in view — and answers with numbers produced by deterministic code, with sources and timestamps. For important decisions, Kestrel can convene an **investment council**: three specialists and a chair who assess, challenge and synthesise. A **paper-trading** mode, backed by an Alpaca paper account, lets the user and Kestrel practise with fake money under explicit approval and limits.

0	0	≈38	3 + 1	2	~600
runtime deps (server)	API keys for market data	agent tools	council specialists + chair	books: REAL · PAPER	automated checks

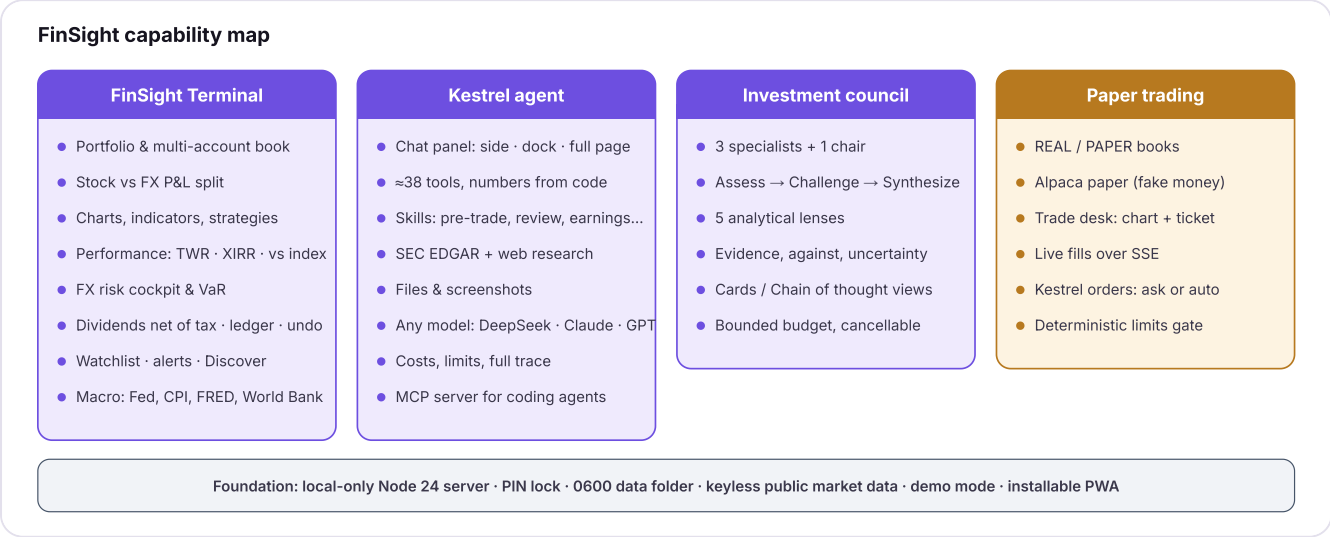
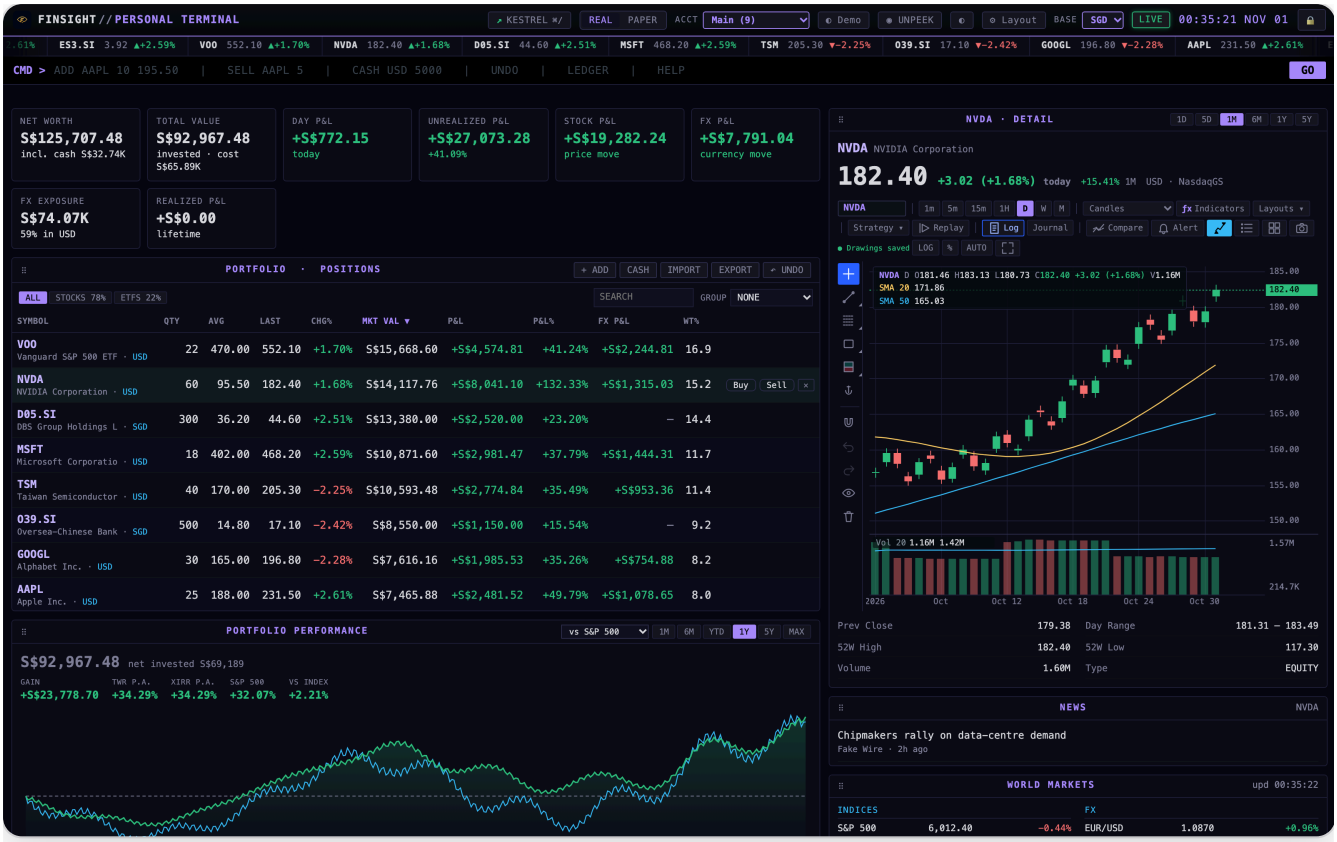
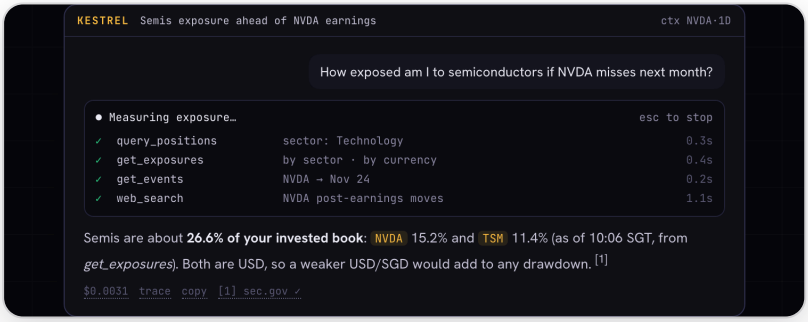


Figure 1 — Capability map



LIVE CAPTURE Screen 1 — The FinSight Terminal running the demo portfolio: summary cards with the stock/FX P&L split, live positions, the NVDA chart and performance vs the S&P 500



GUIDE RECONSTRUCTION Screen 2 — Kestrel answering a portfolio question: each tool call is shown, the figures come from tools, and the answer carries an as-of time, sources and its cost

2. Problem and motivation

2.1 The problem

A retail investor who **funds in one currency and invests in another** (for example converting SGD to buy US stocks) faces four recurring problems:

1. **Hidden currency effect.** Broker screens report gains in one currency. The investor cannot see how much of a gain or loss was the share price and how much was USD/SGD moving.
2. **Fragmented view.** Holdings sit across brokers and accounts (IBKR, moomoo, Tiger, SGX). No single local view rolls them up in the base currency with true time-weighted and money-weighted returns.
3. **Expensive or privacy-hostile tools.** Professional terminals are costly; free portfolio sites require uploading holdings to a third party.
4. **Untrustworthy AI help.** General chatbots invent numbers, do arithmetic badly, forget the user's rules and cannot see the actual book.

2.2 Motivation and vision

FinSight's vision is "**a trading floor for one, running entirely on this machine**": the depth of a professional terminal, the privacy of a local app, and an AI analyst that is grounded in the user's real data and rules and that shows its work.

2.3 Goals

ID	Goal	Measure of success
G1	See the whole book in one base currency	All accounts roll up; every holding shows stock P&L and FX P&L
G2	Make currency risk explicit	Blended entry rate, FX P&L, what-if shock and 1-week VaR available
G3	Stay private and free	No account, no telemetry, keyless data, data folder outside the repo
G4	Ground AI answers in facts	Every figure comes from a tool; answers cite asOf/sources; cost visible
G5	Support better decisions, not automate them	Council gives structured views; the user decides every real trade
G6	Allow safe practice	Paper book with approval/limits; no live-trading path in code

2.4 Non-goals

- Placing real-money orders with any broker. FinSight records the real book; it never sends real orders.
- Multi-user or hosted SaaS operation. FinSight is single-user and local.
- Licensed, real-time market data. Public endpoints are delayed and best-effort.

- Investment advice. Outputs are research aids; the user is the decision-maker.

3. Product overview

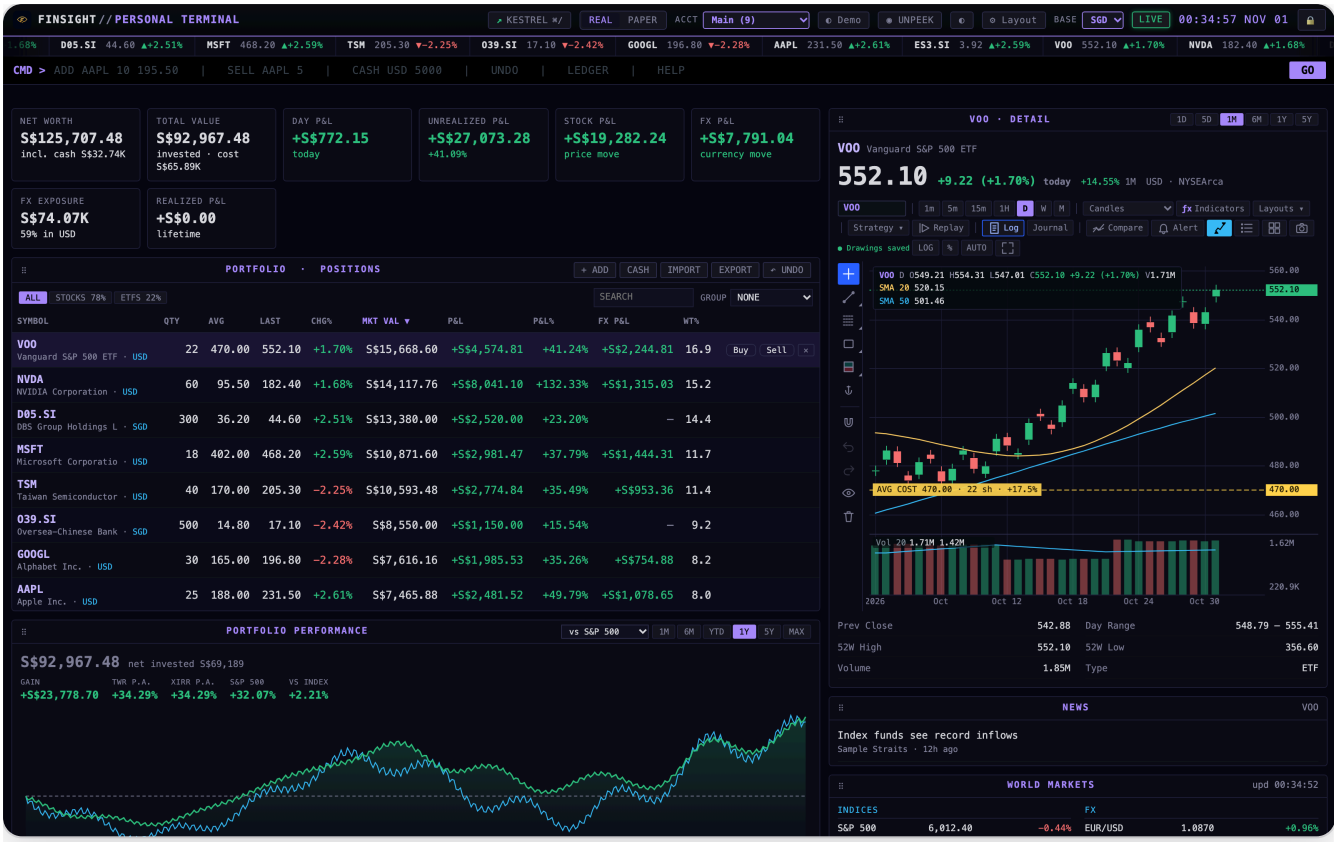
3.1 FinSight Terminal

A local web application (served by `node server.ts` on port 8000) with a dark, themeable terminal UI. Key areas:

Area	What the user gets
Top bar	↗ KESTREL (⌘/), REAL/PAPER switch, account picker (All Accounts / + New), Profile, UNPEEK privacy mode, light/dark, ⚙ Layout, base currency, live status, lock
Command bar	Bloomberg-style commands: `ADD`, `SELL`, `DEL`, `CASH`, `UNDO`, `LEDGER`, `Q`, `WATCH`, `ALERT`, `NEWS`, `WHT`, `MKT`, `HELP`
Summary cards	Net worth, total value, day P&L, unrealized P&L, **stock P&L**, **FX P&L**, FX exposure, realized P&L
Positions	Live table with weight and FX P&L, filters, grouping, Buy/Sell trade sheet, import/export, undo
Security detail	Canvas chart: candle types, indicators (SMA/EMA/Bollinger/RSI/MACD/volume), strategies, replay, journal, compare, drawing tools, chart alerts
Analytics	Performance (TWR, XIRR, vs S&P 500 / MSCI World / All-World / Nasdaq / STI in base currency), net-worth breakdown, allocation donut, sector heatmap
FX Risk	USD exposure, blended entry, spot drift, FX P&L, per-buy FX history, what-if slider, 1σ/95% weekly VaR, target exposure, convert now vs later
Income & history	Dividends net of withholding tax (yield on cost, next ex-date), recent activity, ledger with realized gains by year
Watch & alert	Watchlist, price/trend/RSI/MA alerts with desktop notification, toast and beep
Discover	Movers among held/watched symbols, US market movers (Yahoo screens), US IPO calendar (Nasdaq), upcoming earnings
Macro	World markets, econ calendar (next Fed decision and dot plot, CPI/PCE/jobs/GDP), FRED series, World Bank indicators, inflation gauges
Personalisation	Movable panels, accent palettes (Violet/Amber/Teal/Phosphor), Mono/Sans/Serif fonts, light mode

3.1.1 The terminal, filled with the demo portfolio

The screens below come from the running app with the synthetic demo book (9 holdings across USD and SGD, fake market data). No real holdings are shown.



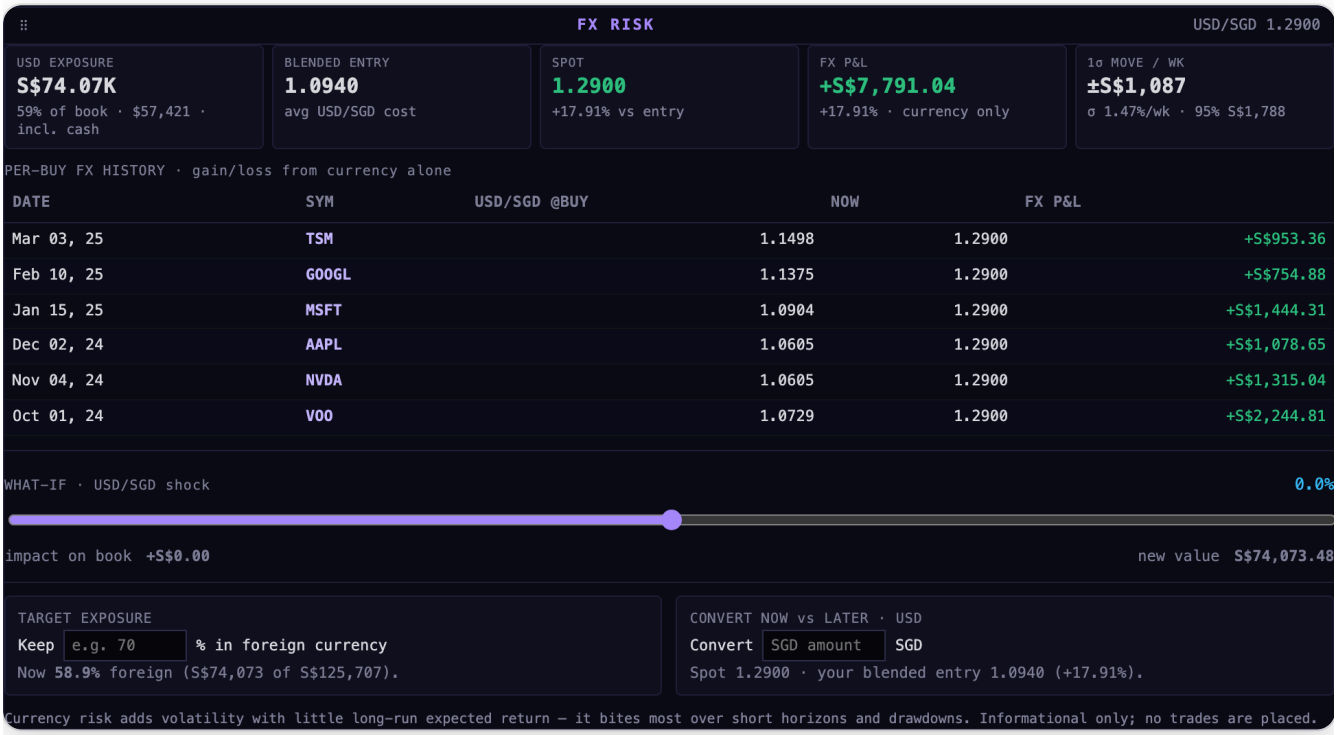
LIVE CAPTURE Screen 3 — Terminal home: top bar (Kestrel, REAL/PAPER, account, UNPEEK, layout, base currency), ticker, command bar, summary cards and positions



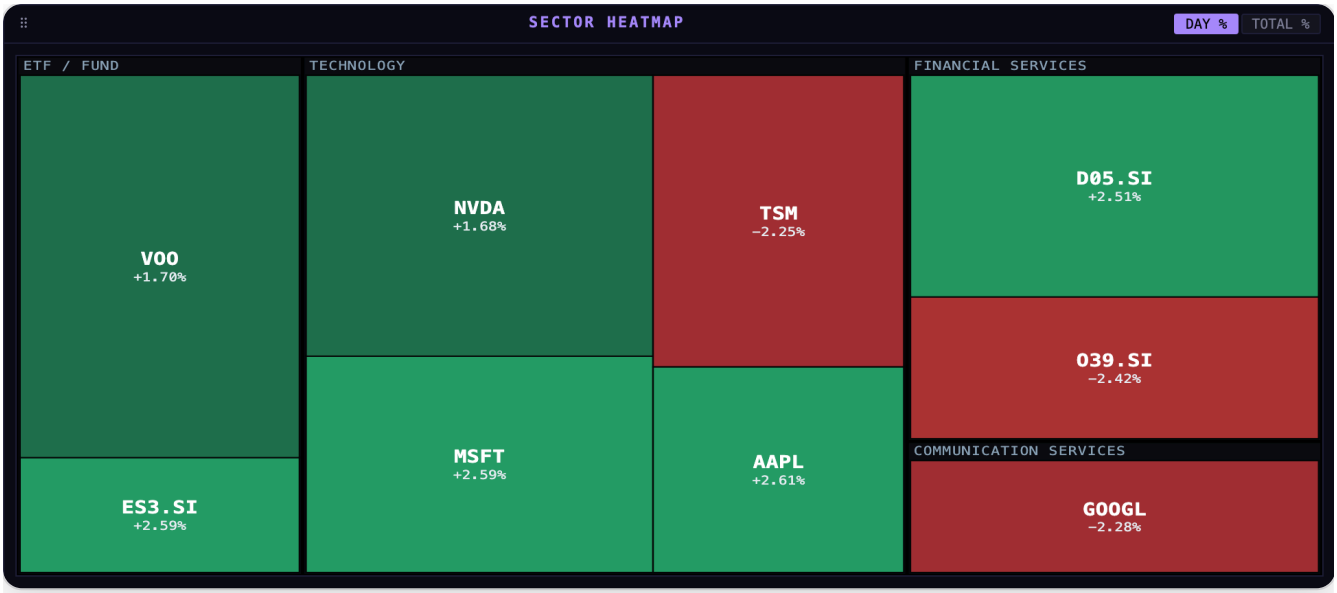
LIVE CAPTURE Screen 4 — Positions: quantity, average cost, last price, market value in SGD, P&L, P&L %, FX P&L and weight per holding



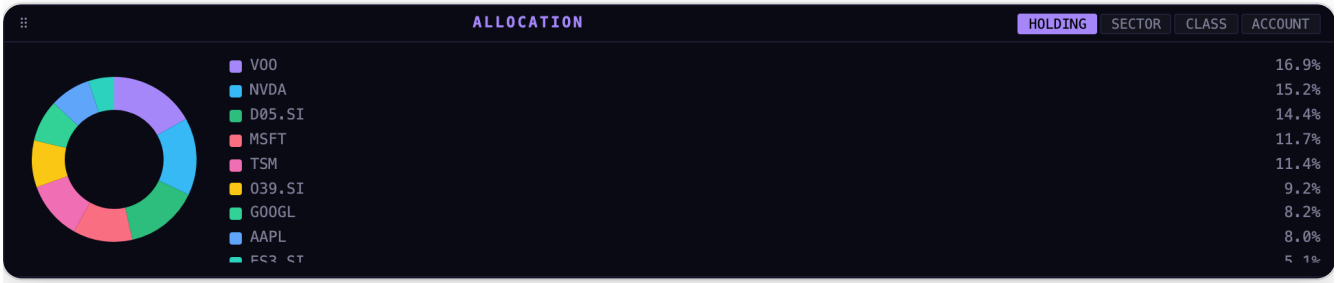
LIVE CAPTURE Screen 5 — Performance: time-weighted and money-weighted return against the S&P 500, in the base currency



LIVE CAPTURE Screen 6 — FX Risk: USD exposure, blended entry rate, spot drift, FX P&L, per-buy history, a what-if shock slider and conversion planning



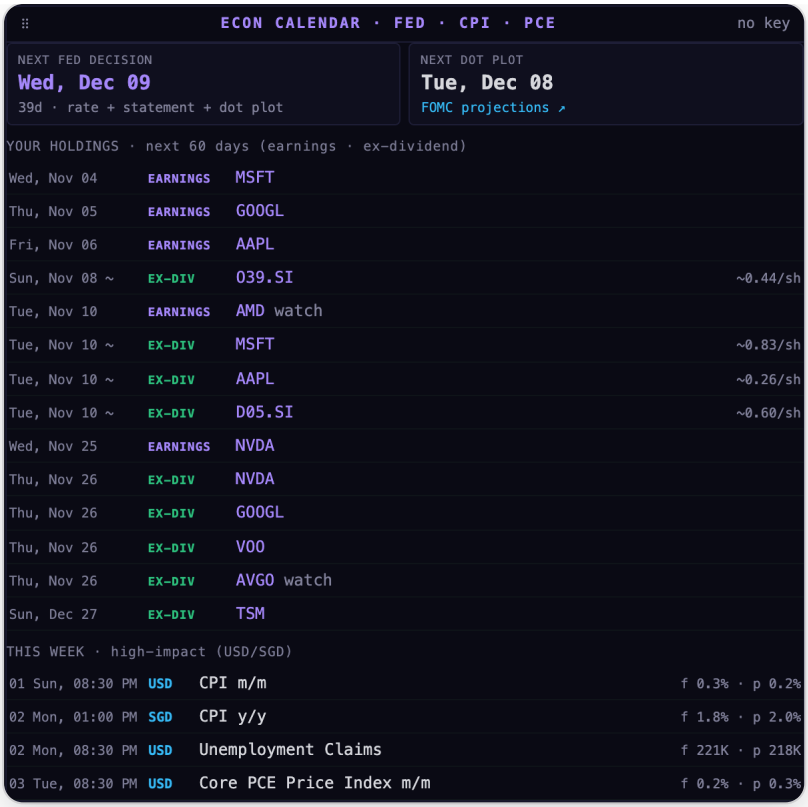
LIVE CAPTURE Screen 7 — Sector heatmap: tile size is position weight, colour is the day's move



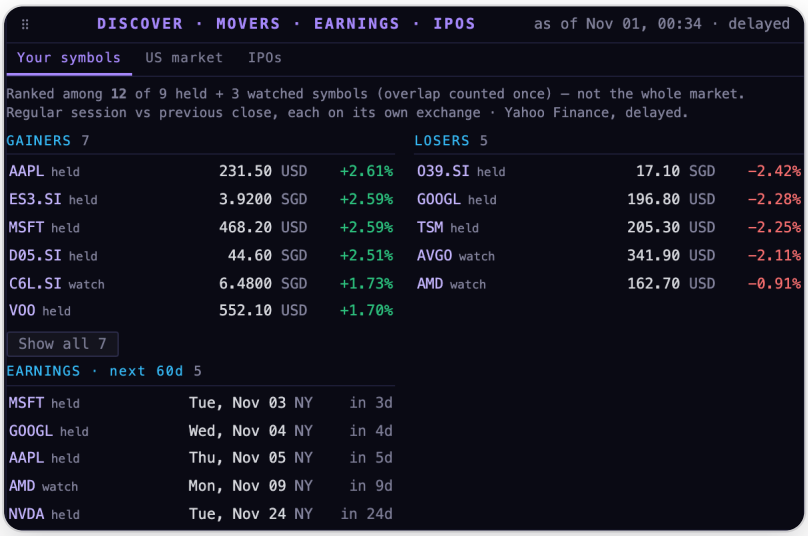
LIVE CAPTURE Screen 8 — Allocation by holding (switchable to sector, asset class or account)



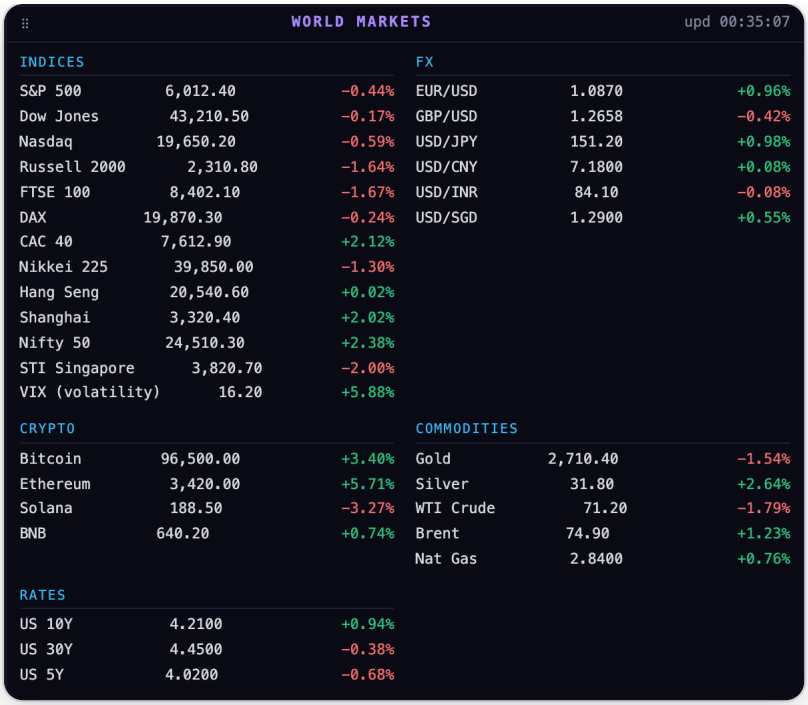
LIVE CAPTURE Screen 9 — Dividends: forward income net of withholding tax, yield on cost and the next ex-date



LIVE CAPTURE Screen 10 — Econ calendar: next Fed decision and dot plot, holdings' earnings and ex-dividend dates, this week's high-impact releases



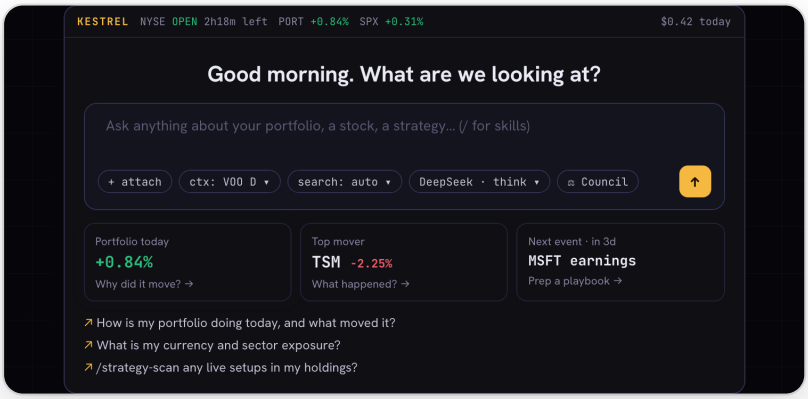
LIVE CAPTURE Screen 11 — Discover: movers among held and watched symbols, and upcoming earnings



LIVE CAPTURE Screen 12 — World markets: indices, FX, crypto, commodities and rates

3.2 ↗ Kestrel — AI research agent

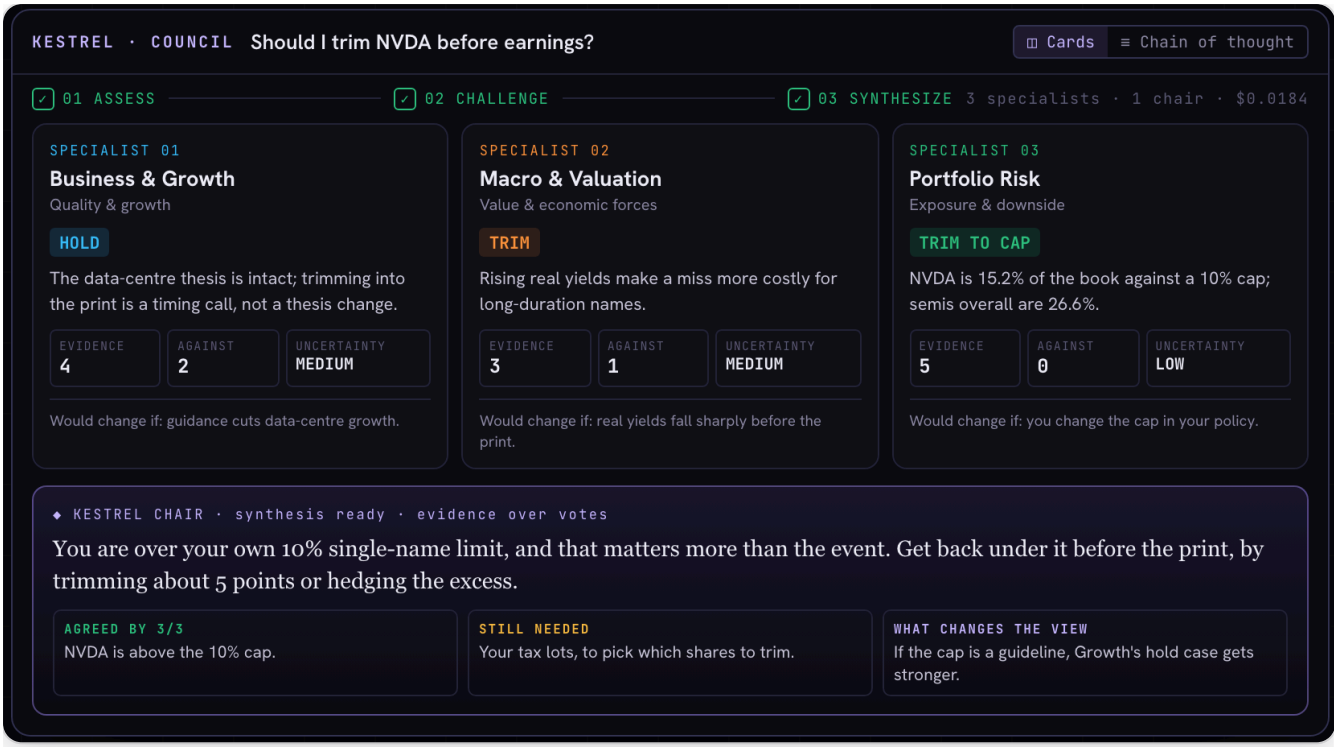
A supervised local service (port 8100) behind FinSight's PIN. It chats in a panel that can be a side drawer, a docked terminal panel, a full page with a workspace column, or a collapsed capsule. It uses ≈38 tools for portfolio facts, strategies, SEC EDGAR, web research, files, records and (in paper mode) paper orders. Skills (`/pre-trade-check` , `/position-review` , `/earnings-review` , `/why-did-it-move` , `/strategy-scan`) run repeatable playbooks. It works with DeepSeek (default), Claude, OpenAI-compatible models or the Claude Code CLI, shows cost per answer and keeps a full trace.



GUIDE RECONSTRUCTION Screen 13 — Kestrel start screen: the ask box with attach, context, search mode, model and ⚖ Council, plus briefing tiles built from the book

3.3 Investment council

An explicit mode (⚖ Council or `/committee`) in which **Business & Growth**, **Macro & Valuation** and **Portfolio Risk** specialists assess a decision independently, run a challenge round, and a **Kestrel Chair** synthesises what they agree on, what is still needed and what would change the view. Five lenses (Balanced, Growth, Speculative/Catalyst, Hedge Fund, Macro) tune emphasis without changing rules or data.



GUIDE RECONSTRUCTION Screen 14 — Council result, Cards view: each specialist's verdict, evidence and uncertainty, with the chair's synthesis below

3.4 Paper trading

A second book next to the real one. The user connects an Alpaca **paper** key pair; FinSight mirrors Alpaca's fills into a paper portfolio so every existing view works. A trade desk (chart, docked ticket, positions/orders/history/Kestrel/settings tabs) places market and limit orders. Kestrel can place paper orders too: **Ask me first** (pending ticket to approve) or **Autonomous**, inside optional deterministic limits.



LIVE CAPTURE Screen 15 — Paper trade desk after real fake-broker orders: account strip, AAPL chart with buy/sell fill markers and an open limit line, the docked ticket and four paper positions

3.5 Demo mode

`npm run demo` starts an isolated FinSight and Kestrel on port 8003 with a synthetic portfolio (PIN 2468) in `~/FinSightDemo`. It never touches real data and is the recommended environment for demos and UAT.

4. Stakeholders and personas

Persona	Description	Primary needs
Owner-investor (primary)	Technically capable individual investor in Singapore, funds in SGD, holds US/SG/HK equities and ETFs across brokers	One private view; FX clarity; grounded research; rules enforced; safe practice
Demo audience	Friends, peers or reviewers watching a walkthrough	Understand value quickly without seeing real holdings (demo mode, UNPEEK)
Contributor / coding agent	Developer or AI coding agent (Claude Code, Codex, Cursor) changing the code	Clear specs, boundaries, quality gates, docs-as-code process (AGENTS.md)
UAT tester	Human or Codex computer-use agent executing acceptance scripts	Deterministic environment (demo + fake market), precise steps and expected results (Section 20)
External MCP client	Claude Code / Codex / Cursor using FinSight tools over MCP	Same read-only tools and citation rules as Kestrel

> **Single-user by design**
>
> There is exactly one user class with full authority. There is no multi-tenancy, no roles and no remote access. All security controls assume an attacker is another process, web page or document on the same machine — not a second legitimate user.

5. User stories

Stories follow "As a ... I want ... so that ..." with acceptance criteria (AC). IDs map to functional requirements (Section 7) and UAT cases (Section 20).

Epic A — Access and setup

ID	Story	Acceptance criteria
US-A1	As the owner, I want to lock FinSight behind a PIN so that nobody using my computer can read my portfolio.	PIN required for all `/api/*` data; wrong PIN shows "Wrong PIN"; repeated failures are throttled; 🔒 locks immediately
US-A2	As the owner, I want to choose REAL or PAPER before unlocking so that I land in the right book.	Lock screen offers both; choice persists across tabs and reloads; PAPER is visibly marked
US-A3	As a new user, I want a guided first-run so that I can start in minutes.	Operator card asks name and base currency; offers CSV import, manual add or empty start; reopen from Profile
US-A4	As a presenter, I want a demo mode so that I can show FinSight without exposing my data.	`npm run demo` opens port 8003 with synthetic data and PIN 2468; real data folder untouched

Epic B — Portfolio and analytics

ID	Story	Acceptance criteria
US-B1	As the owner, I want every holding valued in my base currency so that I see one total.	Summary cards and positions use BASE; changing BASE re-prices everything
US-B2	As the owner, I want gains split into stock and FX parts so that I know what drove returns.	Unrealized = Stock P&L + FX P&L; per-position FX P&L column; lots store purchase FX
US-B3	As the owner, I want to record buys and sells quickly so that the book stays accurate.	Command bar and trade sheet record trades; sheet previews position after and new average; UNDO reverses
US-B4	As the owner, I want to import broker statements so that I don't retype holdings.	Plain CSV, IBKR, moomoo, Tiger supported; IBKR fills previewed with duplicate detection
US-B5	As the owner, I want multiple accounts and transfers so that the book matches reality.	Create accounts; All Accounts view; transfers keep lots, cost and FX; undoable
US-B6	As the owner, I want true performance vs an index so that I know if I'm adding value.	TWR, XIRR, gain, vs index in base currency for 1M... MAX; benchmark selectable
US-B7	As the owner, I want to see FX risk so that I can plan conversions.	FX Risk shows exposure, blended entry, spot drift, VaR, what-if, target, convert-now
US-B8	As the owner, I want dividend income after tax so that I can plan cash flow.	Forward income net of WHT; yield on cost; next ex-date; `WHT` command changes rates

Epic C — Markets, discovery and alerts

ID	Story	Acceptance criteria
US-C1	As the owner, I want a full chart for any symbol so that I can analyse price action.	Click a row or `Q SYM`; timeframes, chart types, indicators, strategies, drawings saved
US-C2	As the owner, I want price alerts so that I don't have to watch the screen.	`ALERT AAPL > 320` or UI; fires notification, toast and beep
US-C3	As the owner, I want to see what's moving and what's coming so that I can prepare.	Discover tabs (Your symbols, US market, IPOs) with source, as-of and stale flags; rows open detail
US-C4	As the owner, I want the macro calendar so that I know when rates news lands.	Next Fed decision/dot plot always shown; this week's high-impact releases; holdings' earnings/ex-dates
US-C5	As a presenter, I want to hide money figures so that I can share my screen.	UNPEEK blanks every money figure including Kestrel/council views

Epic D — Kestrel agent

ID	Story	Acceptance criteria
US-D1	As the owner, I want to ask questions about my portfolio in plain language so that research is fast.	⌘/ opens Kestrel; answers stream; figures come from tools with asOf; sources cited
US-D2	As the owner, I want to see what Kestrel did so that I can trust or challenge it.	Tool calls shown live; per-answer cost; trace of every model/tool/data call
US-D3	As the owner, I want repeatable playbooks so that important checks are never skipped.	`/` menu lists skills; `/pre-trade-check` sizes from real cash and checks concentration and events
US-D4	As the owner, I want to choose the model and thinking effort so that I balance cost and depth.	Model picker and effort menu; `/model`; cost per answer and per day visible
US-D5	As the owner, I want spending limits so that a runaway loop can't cost much.	Per-turn and per-day caps editable (`/limits`); limits enforced before billable calls
US-D6	As the owner, I want to give Kestrel files and screenshots so that it can analyse them.	Attach/paste/drop PDFs, DOCX, CSV, images; content treated as untrusted data
US-D7	As the owner, I want Kestrel to remember my rules only with my consent so that records stay mine.	Stated facts become `propose_record_update` proposals; nothing changes until confirmed
US-D8	As a developer, I want FinSight tools in my coding agent so that I can query data from my IDE.	`claude mcp add finsight -- node agent/src/cli.ts mcp` exposes the registry over stdio

Epic E — Council

ID	Story	Acceptance criteria
US-E1	As the owner, I want several expert views on a decision so that I avoid one-sided reasoning.	Council runs 3 specialists + chair through Assess → Challenge → Synthesize

ID	Story	Acceptance criteria
US-E2	As the owner, I want to set an analytical lens so that the review fits my question.	Lens picker (5 lenses) shown when Council is on; lens recorded with the turn
US-E3	As the owner, I want to see evidence and uncertainty so that I can judge each view.	Cards show verdict, evidence/against counts, uncertainty, would-change-if; Chain of thought view
US-E4	As the owner, I want the council to admit gaps so that I'm not misled.	Ambiguous company or missing data → clarification/gap, partial status; unverified figures flagged

Epic F — Paper trading

ID	Story	Acceptance criteria
US-F1	As the owner, I want a paper book so that I can practise without risking money.	PAPER mode with Alpaca paper keys; real book untouched; manual book edits refused
US-F2	As the owner, I want a professional trade desk so that placing orders is quick and clear.	Account strip, chart linked to ticket, BUY/SELL, Market/Limit, shares or US$, quick sizes, TIF, estimate
US-F3	As the owner, I want fills to appear live so that I don't refresh.	trade_updates → book resync → SSE; positions, ledger, P&L, chart markers update
US-F4	As the owner, I want Kestrel to propose or place paper orders under my control so that I can test agentic trading safely.	Ask mode creates pending tickets (Approve/Reject); Auto places directly; limits refuse with reason; only the PIN session can change settings

6. User flows

The five flows below cover every primary journey. Section 20 turns each into executable UAT scripts.

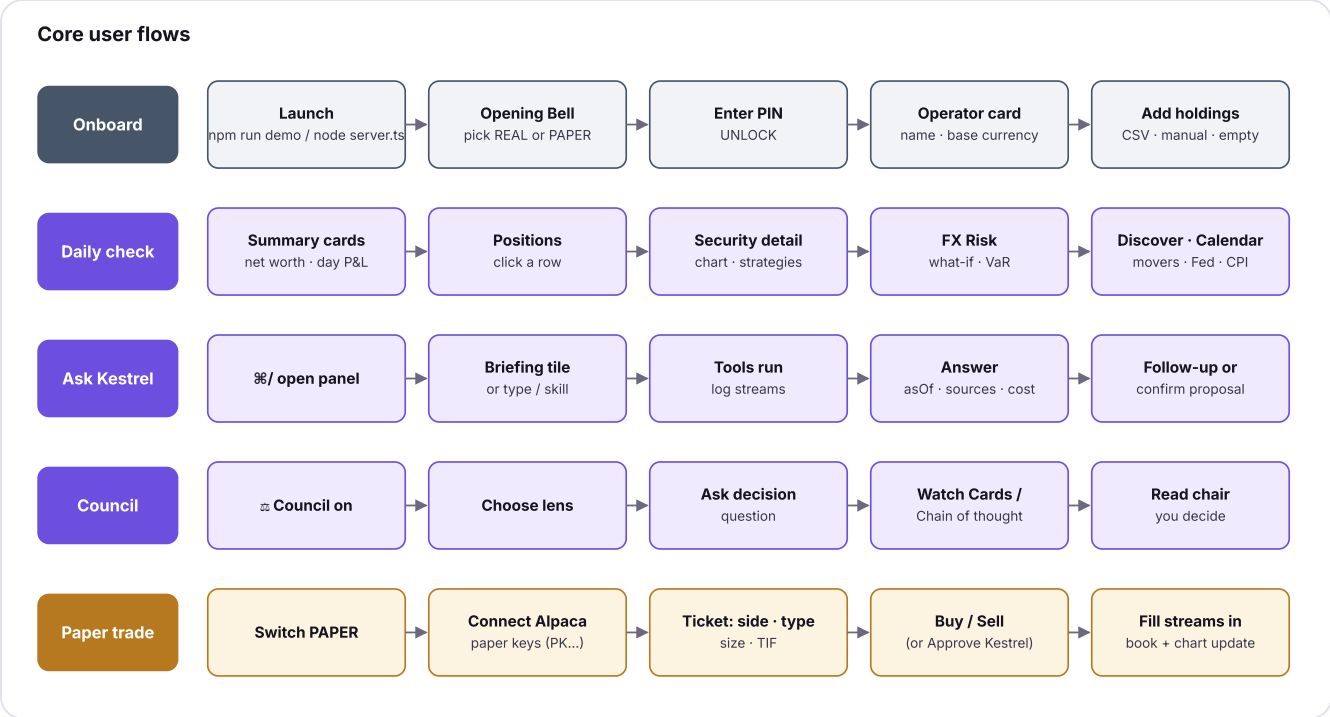


Figure 2 — Core user flows

6.1 Flow narratives

Flow	Entry point	Happy path	Key alternate paths
Onboard	Launcher or `node server.ts`	Opening Bell → choose book → PIN → operator card → import → terminal	Wrong PIN (throttled) · start empty · reopen via Profile · forgot PIN (delete auth.json)
Daily check	Terminal home	Summary cards → positions → click row → chart → FX Risk → Discover → calendar	Stale quotes flagged · UNPEEK for sharing · switch account or base
Ask Kestrel	⌘/ or ↗ KESTREL	Briefing tile or question → tool log → answer with asOf, sources, cost	Agent offline hint · stop with Esc · limits reached · proposal confirm/dismiss
Council	⚖ Council pill or `/committee`	Lens → decision question → cards/chain stream → chair synthesis	Ambiguous subject → clarification · budget refusal · cancel aborts tree
Paper trade	PAPER (lock screen or top bar)	Connect keys → ticket → submit → fill streams → book updates	Live keys refused · limit refusal · Kestrel ticket approve/reject · cancel open order

7. Functional specification

Requirements use MUST for mandatory behaviour. Each FR traces to stories (Section 5) and tests (Appendix D).

7.1 Access, session and modes

ID	Requirement
FR-ACC-1	The server MUST gate every data API behind a PIN session; the UI shell may load without it.
FR-ACC-2	PINs MUST be stored scrypt-hashed in `auth.json`; failed attempts MUST be throttled.
FR-ACC-3	Session cookies MUST be `HttpOnly; SameSite=Strict`; idle lock after `LOCK_IDLE_MIN` minutes when configured.
FR-ACC-4	The book mode (REAL/PAPER) MUST be stored in `book.json` and apply to every tab and to Kestrel.
FR-ACC-5	First run MUST offer the operator card (name, base currency, CSV/manual/empty).

7.2 Portfolio, trades and imports

ID	Requirement
FR-PF-1	Positions MUST be lot-based (`q, px, fxUsd, t`), preserving purchase-time FX for return decomposition.
FR-PF-2	Unrealized P&L MUST equal Stock P&L + FX P&L in the base currency.
FR-PF-3	Buy/sell/edit/delete/cash/import/transfer MUST be undoable (25-step undo stack).
FR-PF-4	The trade sheet MUST support new symbols, show position-after and new average before saving.
FR-PF-5	Imports MUST accept plain CSV and IBKR, moomoo and Tiger statements; IBKR fills MUST be previewed with duplicate detection.
FR-PF-6	Transfers between accounts MUST preserve lots, cost basis and FX; the ledger MUST report realized gains by year and export CSV.
FR-PF-7	Back-dated trades MUST fetch the historical FX rate for the trade date.

7.3 Market data, chart and analytics

ID	Requirement
FR-MD-1	Quotes, history, FX, search and news MUST come from keyless public sources (Yahoo Finance) with stale-while-revalidate caching and 429 backoff.
FR-MD-2	The chart MUST support 1m–M intervals, six chart types, indicators, strategy overlays, replay, journal, compare, drawings and alerts.
FR-AN-1	Performance MUST replay actual trades with historical prices and FX to compute TWR and XIRR, and compare against a chosen index in base currency.

ID	Requirement
FR-AN-2	FX Risk MUST show exposure, blended entry, spot drift, FX P&L, per-buy history, what-if shock, 1σ and 95% weekly VaR, target exposure and convert-now estimate.
FR-AN-3	Dividends MUST be forward-estimated net of per-market withholding (default US 30%), with yield on cost and next ex-date.
FR-DS-1	Discover MUST label its universe, baseline, session, as-of and stale/failed states; US movers via Yahoo screens; IPOs via Nasdaq with EDGAR links.
FR-MC-1	The econ calendar MUST always show the next FOMC decision and dot-plot date, plus this week's high-impact releases.
FR-AL-1	Alerts MUST fire desktop notification, toast and beep for price, trend line, RSI and moving-average rules.

7.4 Kestrel agent

ID	Requirement
FR-K-1	Kestrel MUST run as a supervised loopback service reachable only through FinSight's `/api/agent/*` proxy.
FR-K-2	All totals, weights and sizes MUST come from tools (`query_positions` , `get_exposures` , `size_position` ...), never model arithmetic.
FR-K-3	Answers MUST cite `asOf` / `retrievedAt` for data and source URL + date for web facts; missing data MUST be reported.
FR-K-4	Kestrel MUST stream tool calls and text over SSE with event ids for replay; Esc/■ MUST stop a turn.
FR-K-5	Users MUST be able to switch model and thinking effort; each answer MUST show its cost; `/cost` reports spend.
FR-K-6	Limits (defaults: 25 steps, 300 s, US$0.25 per turn, US$5 per day, 20 searches, 25 fetches, 8 council research calls) MUST be enforced and editable.
FR-K-7	Skills MUST load from `agent/skills/*/SKILL.md` and user skill folders; `/` lists commands and skills.
FR-K-8	Durable facts the user states MUST become `propose_record_update` proposals; Kestrel MUST NOT edit records directly.
FR-K-9	Attachments (PDF, DOCX, CSV, images) MUST be readable and treated as untrusted data.
FR-K-10	The tool registry MUST be exposed over MCP stdio with the same read-only semantics.
FR-K-11	Chats MUST support new, clear, delete (with confirmation), rename, pin, search and bulk clear.

7.5 Council

ID	Requirement
FR-C-1	Council MUST be explicit (toggle or `/committee`), never automatic.
FR-C-2	It MUST run three specialists, one challenge call each and a separate chair, with concurrency two.

ID	Requirement
FR-C-3	Specialists MUST use role-scoped tools; paid web search/fetch excluded in this release.
FR-C-4	Each assessment MUST validate against the versioned contract (verdict, evidence, against, uncertainty, invalidation).
FR-C-5	Cost MUST be reserved before each call; a quarter of the turn ceiling is held for synthesis; cancellation aborts the tree.
FR-C-6	Ambiguous subjects or unavailable data MUST yield clarification or gaps, never chart-context substitution.

7.6 Paper trading

ID	Requirement
FR-P-1	Only Alpaca's **paper** base URL MAY be used; keys MUST be verified against it and live keys refused.
FR-P-2	Keys MUST be stored 0600 in `alpaca-paper.json` (or env) and never sent to the browser or agent.
FR-P-3	The paper book MUST mirror Alpaca fills into `paper/portfolio.json`; manual book edits MUST be refused in PAPER mode.
FR-P-4	Orders: market/limit, buy/sell, day/gtc, whole or fractional shares, or notional USD for market orders; cancel open orders.
FR-P-5	Kestrel tools `paper_account`, `paper_place_order`, `paper_cancel_order` MUST pass the same server gate.
FR-P-6	Approval `ask` (default) MUST create pending tickets; `auto` MUST submit directly; optional limits MUST refuse deterministically with a reason.
FR-P-7	Only the PIN session MAY change paper settings; every order MUST record origin (user/kestrel + thread).

8. Non-functional requirements

Quality	Requirement / target
Performance	Agent overhead before first provider byte p95 ≤ 150 ms; context pack ≤ 500 ms p95 warm; panel open ≤ 100 ms; no N+1 calls to FinSight; idle CPU ≈ 0 (event-driven)
Availability	Terminal works offline from the persisted cache; failed upstream refreshes serve last-known-good with stale flags
Reliability	A crash mid-turn leaves an `interrupted` partial message; the supervisor restarts the agent with backoff; DB integrity check and backup before migrations
Security & privacy	Loopback only; PIN; secrets in Keychain or 0600 files; no telemetry; nothing personal in the repo (Section 16)
Usability	Keyboard command bar, ⌘/ for Kestrel, responsive layout to phone width, installable PWA
Accessibility	Axe checks in CI, accessible names match visible text, keyboard-operable tabs, reduced-motion support, light mode, high-contrast-friendly tokens
Maintainability	TypeScript strict, erasable syntax for native Node execution, module boundaries, specs before code, ADRs for significant decisions
Portability	macOS primary; Windows/Linux supported for the terminal; Keychain behind an interface with env fallback
Cost	DeepSeek default (≈US$0.0003–0.0008 per simple turn); caps per turn and per day
Scalability	Plugins (brains, skills, strategies, search providers) add capability without core edits; versioned contracts

9. System architecture

9.1 Context

FinSight is a local system with one human user. It reaches out to keyless market-data sources, the Alpaca paper API, model providers and research sources. Coding agents can attach to its tools over MCP.

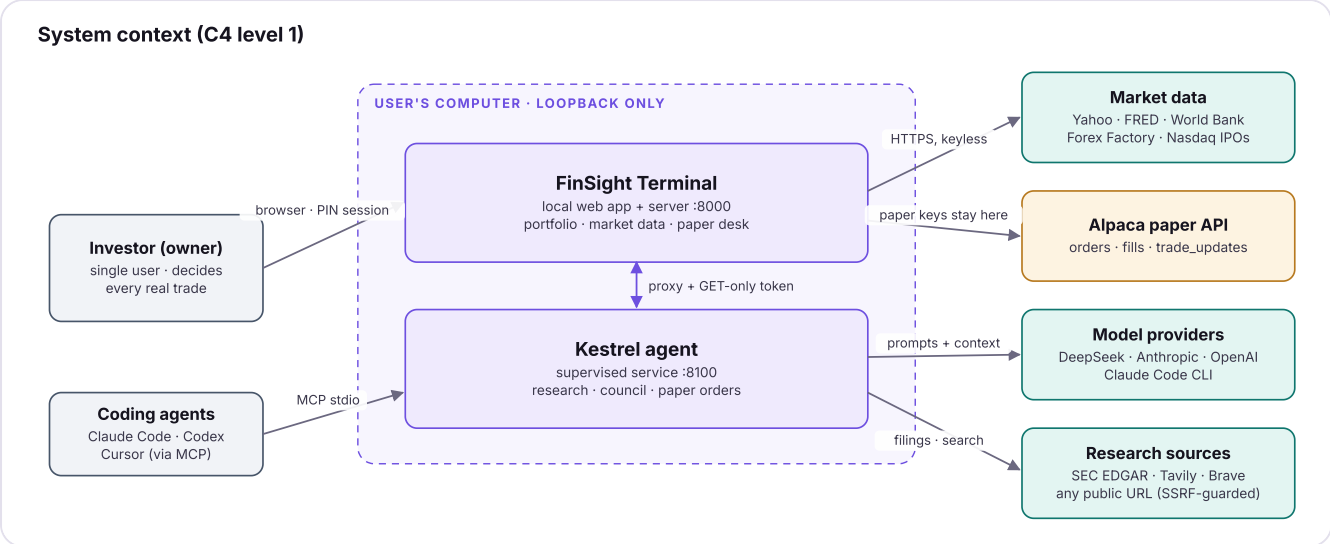


Figure 3 — System context

9.2 Containers and processes

Three processes cooperate on the user's machine: the browser UI, the FinSight server (which also supervises the agent and links to the paper broker) and the Kestrel service. All persistent state lives in `FINSIGHT_DATA_DIR`, outside the repository.

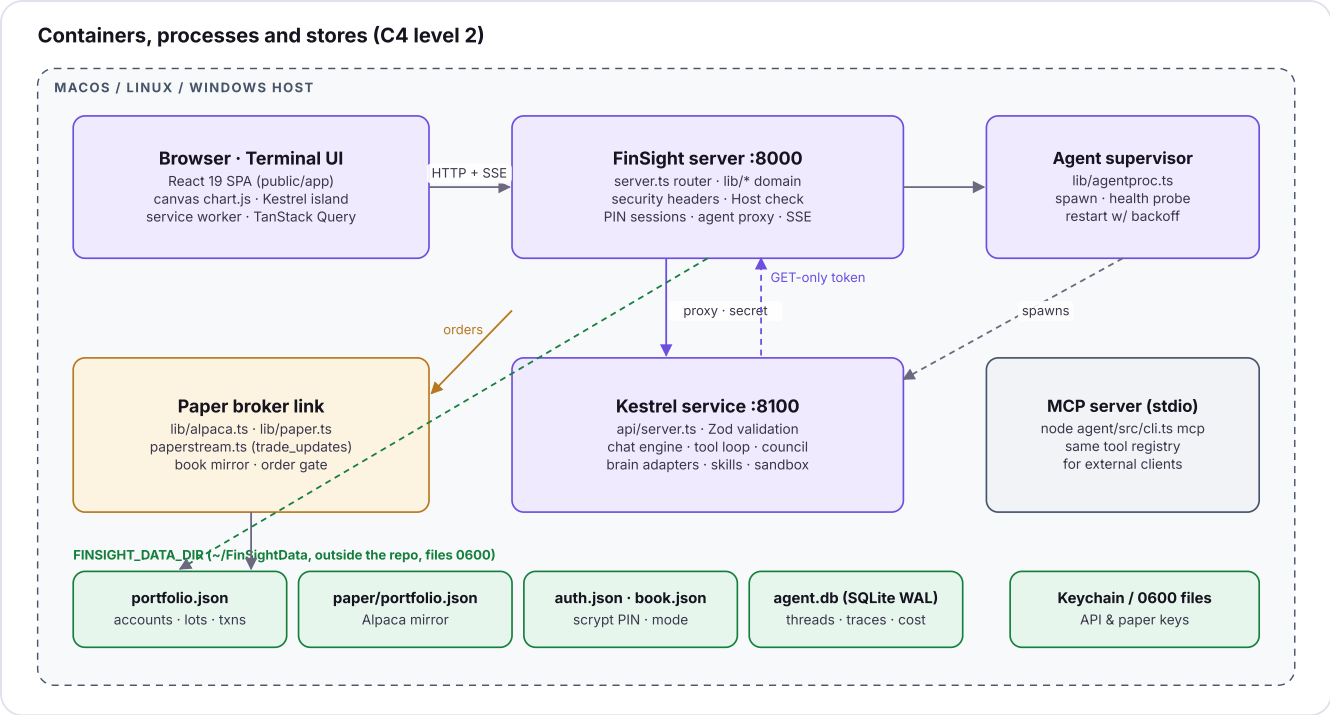


Figure 4 — Containers, processes and stores

Process	Port	Started by	Responsibility
FinSight server	8000 (8003 demo, 8002 standalone)	`node server.ts`, `start.command`, autostart	UI, `/api/*`, sessions, market data, paper link, agent proxy & supervision
Kestrel service	8100 (8103 demo)	FinSight supervisor	Chat engine, tools, council, skills, SQLite store
MCP server	stdio	External client	Exposes the tool registry to Claude Code / Codex / Cursor
Strategy sandbox	child process	Kestrel	Executes strategy engines with timeout and memory caps

9.3 FinSight server components

`server.ts` is only routing, security headers, the session gate and the agent proxy. Domain logic lives in `lib/` modules with single responsibilities.

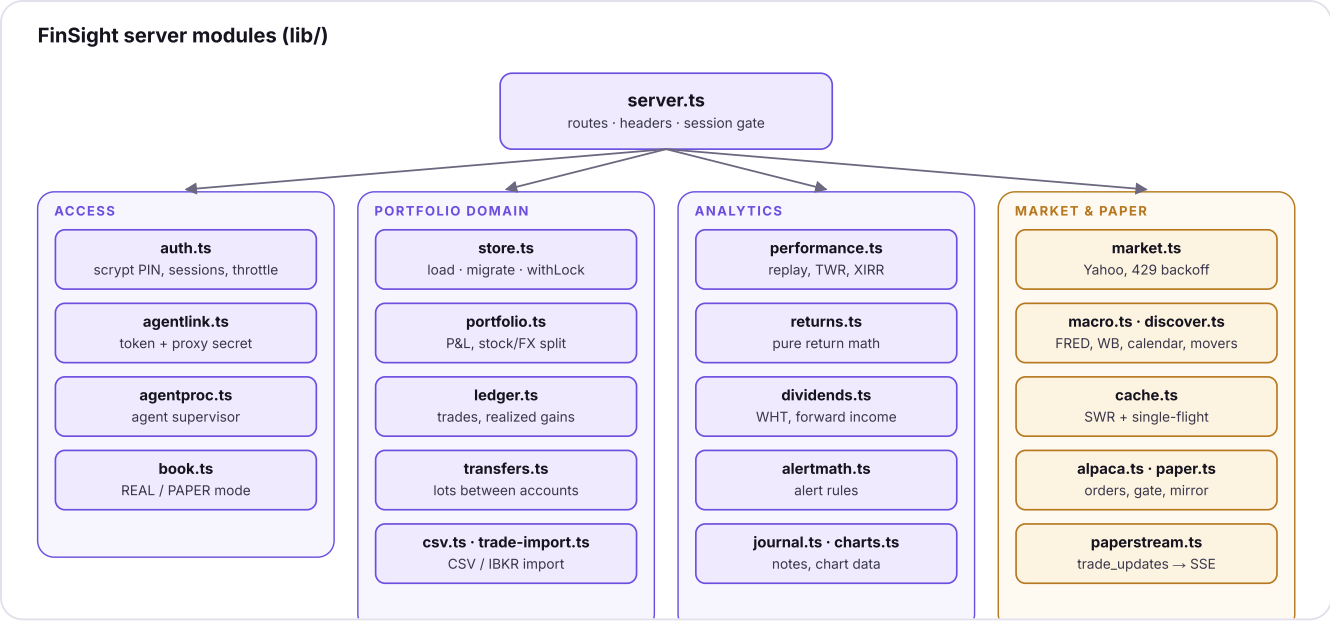


Figure 5 — Server modules

9.4 Frontend architecture

- **React 19 + TypeScript** source in `web/src`, one app (`App.tsx`) composed of feature folders (`features/positions`, `features/kestrel`, `features/paper`, …) loaded as lazy chunks.
- **Build:** esbuild (`scripts/build-web.mjs`) emits committed assets to `public/app/` (bootstrap, main, chunks, manifest) so users need no `npm install` to run.
- **State:** `bootstrap.js` creates `window.FinSight` with a Zustand UI store and refresh bus before first paint; server data via **TanStack Query** with polling suspended while locked or hidden.
- **Chart:** a self-contained canvas engine (`public/chart.js`) plus indicator, strategy and journal helpers in plain JS.
- **Appearance:** tokens on `:root` in `public/styles.css`; `data-theme`, `data-font`, `data-mode` applied before paint.
- **PWA:** service worker precaches the shell from the build manifest; warm cache survives restarts.

9.5 Kestrel agent components

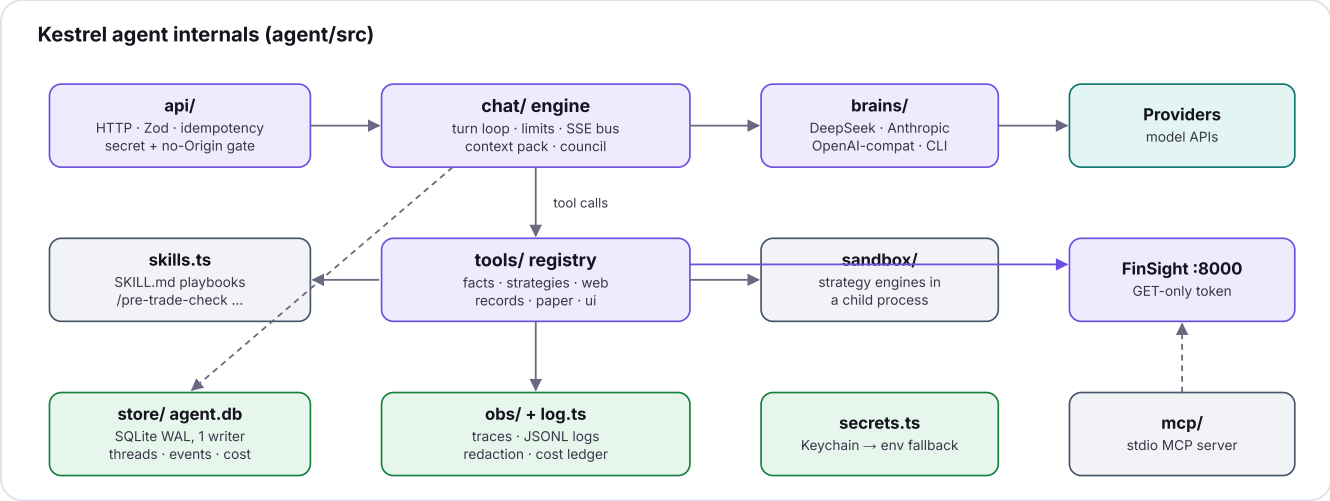


Figure 6 — Kestrel internals

9.6 Key runtime sequences

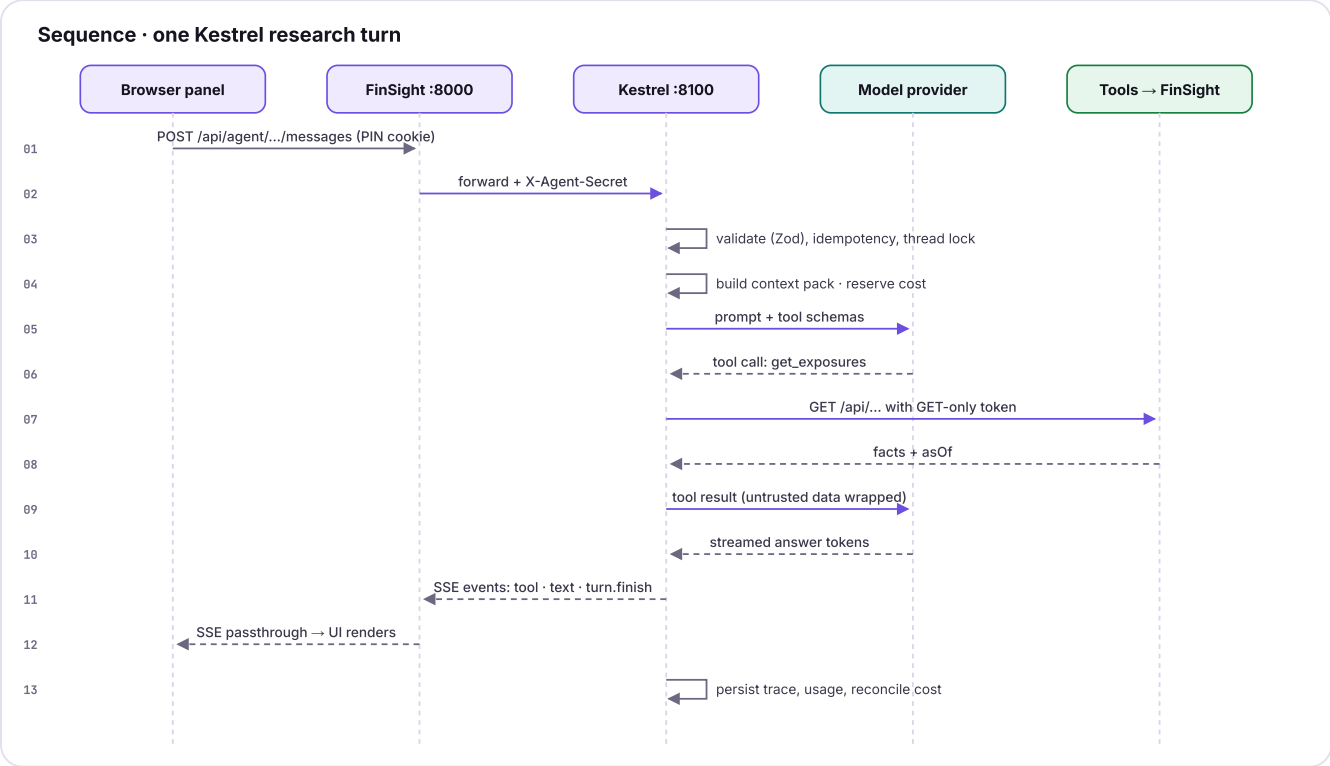


Figure 7 — One Kestrel research turn

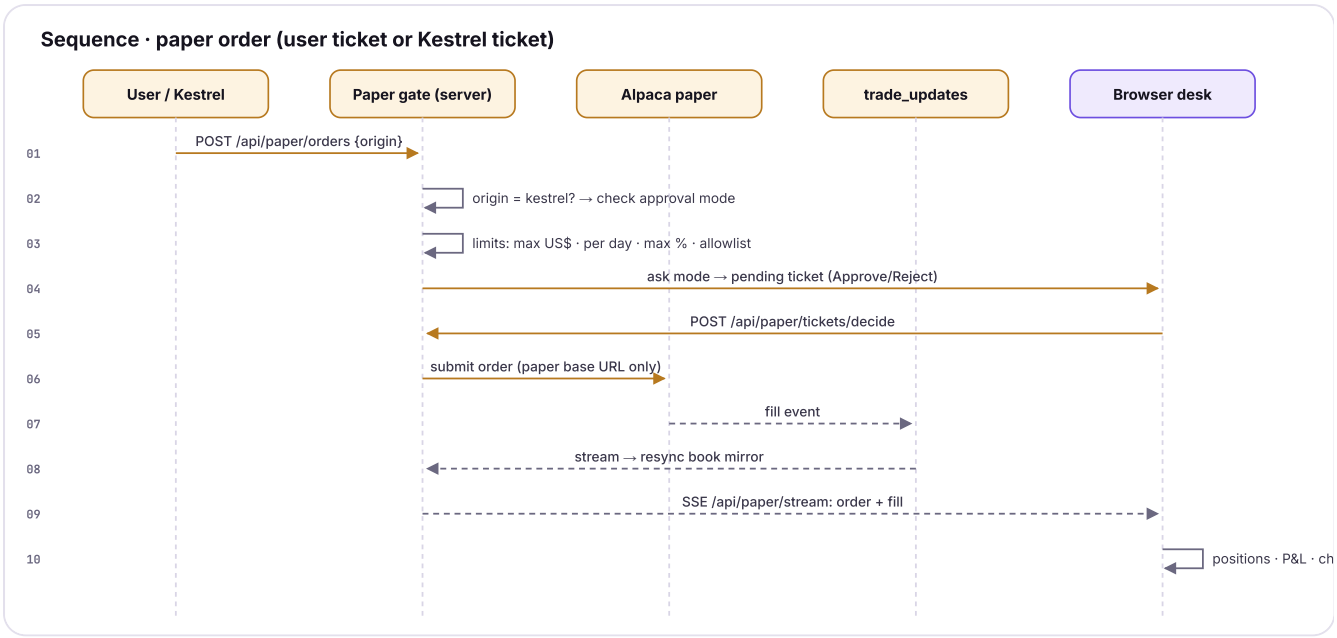


Figure 8 — Paper order through the gate

10. Software specification

10.1 Data model

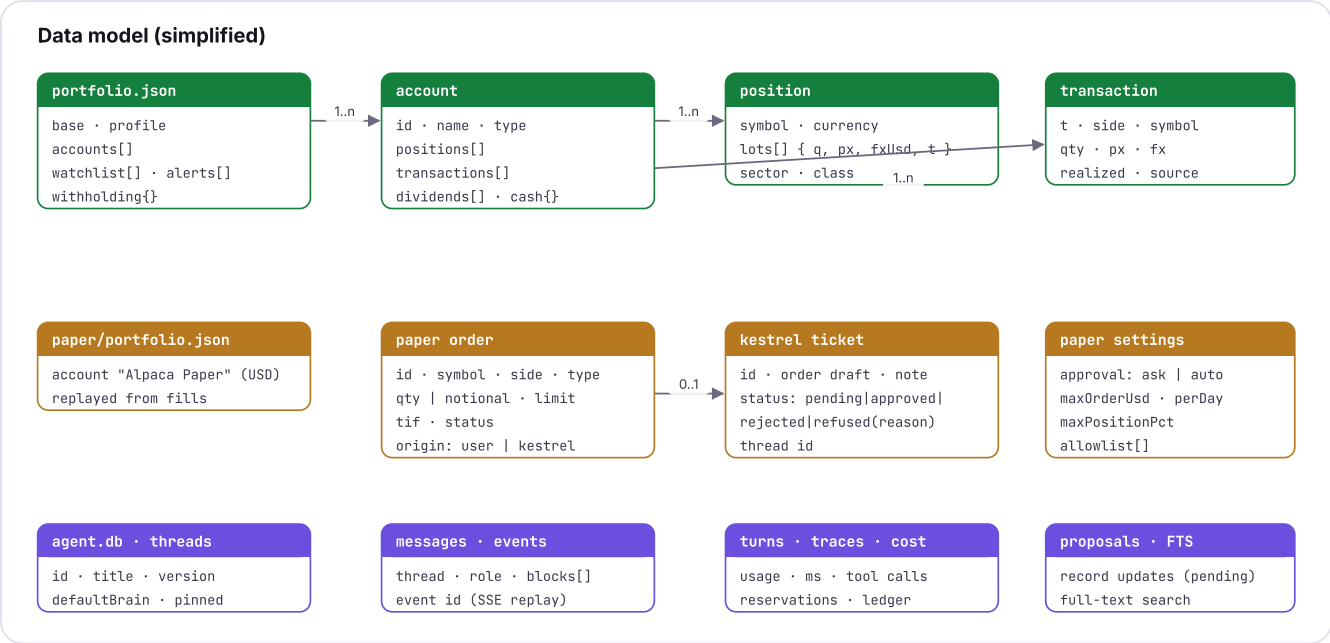


Figure 9 — Data model

File / table	Location	Notes
`portfolio.json`	data dir	Accounts, positions (lots), transactions, dividends, cash, watchlist, alerts; atomic write, 0600
`undo.json`	data dir	25 snapshots for undo
`cache.json`	data dir	Persisted market cache (≤ 30 days)
`auth.json`	data dir	scrypt PIN hash + salt
`book.json`	data dir	REAL / PAPER mode
`paper/portfolio.json`	data dir	Alpaca mirror book
`alpaca-paper.json`	data dir	Paper keys, 0600, never served
`agent-link.json`	data dir	Rotated GET-only token and proxy secret
`agent/agent.db`	data dir	SQLite WAL: threads, messages, events, turns, traces, cost ledger, proposals, FTS
`agent/logs/*.jsonl`	data dir	Interaction log, redacted, 14-day retention
`records/`	data dir	INVESTMENT_POLICY.md, INVESTOR_PROFILE.md and other user records

10.2 HTTP API (selected)

Method & path	Purpose
GET /api/portfolio?base=	Portfolio with quotes, P&L, stock/FX split
POST /api/portfolio/buy · sell · …	Book mutations (session + same-origin, locked, undoable)
GET /api/history · /api/history/batch	OHLCV bars (cached)
GET /api/discover?account=	Movers, earnings, exclusions with basis and asOf
GET /api/events	Earnings and ex-dividend dates
GET /api/agent-status · /api/agent/*	Agent health; session-gated proxy to Kestrel
GET /api/paper/state · settings · stream	Paper account, settings, SSE order/fill stream
POST /api/paper/orders · orders/cancel	Place / cancel paper orders (gate applies)
POST /api/paper/tickets/decide	Approve or reject a Kestrel ticket
POST /api/paper/connect · disconnect · settings · sync	Key management, settings (PIN session only), resync

The full endpoint reference is in `DOCS.md`. Kestrel's own API (`/v1/threads`, `/v1/threads/:id/messages`, `/v1/threads/:id/events`, `/v1/brains`, `/v1/cost`, `/v1/limits`, `/v1/health`) is reachable only through the proxy.

10.3 Agent tool registry

Category	Tools
Portfolio facts	`get_portfolio`, `query_positions`, `portfolio_weights`, `get_exposures`, `get_cash`, `get_accounts`, `get_context_pack`
Market & macro	`get_quote`, `get_history`, `get_earnings_dates`, `get_macro_series`, `get_fund_profile`, `valuation_metrics`
Strategies & sizing	`list_strategies`, `get_strategy`, `run_strategy`, `scan_strategies`, `size_position`
Research	`edgar_filings`, `edgar_financials`, `web_search`, `fetch_url`
Records & memory	`get_policy`, `get_profile`, `get_decisions`, `search_memory`, `propose_record_update`
Attachments	`list_attachments`, `read_attachment`, `search_attachments`
UI	`render_ui` (Intelligent UI views: cards, tables, charts)
Paper (PAPER mode)	`paper_account`, `paper_place_order`, `paper_cancel_order`

10.4 Plugin manifests

Plugin	Location	Defines
Brain	`agent/plugins/brains/<id>/`	Provider, model id, pricing, thinking levels, capabilities (deepseek-flash, deepseek-pro, haiku, claude-code, gpt-6-luna, gpt-6-sol)
Skill	`agent/skills/<name>/SKILL.md`	Agent Skills-format playbook: description, steps, required tools
Strategy	strategy library + `manifest.json`	Card (prose/metadata) and optional mechanical engine run in the sandbox
Search provider	agent config	Tavily, Brave, SearchGPT; chosen per answer

10.5 SSE event model

Kestrel streams typed events (`turn.started` , tool start/finish, text deltas, notices, council stage/assessment events, `turn.finish` with usage and cost). Each event has a monotonically increasing id so a reconnecting client resumes without duplicates. The paper stream (`/api/paper/stream`) carries order and fill events and stream-state changes.

11. Technology stack

Layer	Technology	Why
Runtime	Node.js ≥ 24, native TypeScript (erasable syntax)	No build step for the server; one language end to end
Server	`node:http`, native `fetch`, zero runtime dependencies	Small attack surface, nothing to install
Frontend	React 19.3, TypeScript 7, TanStack Query 5, Zustand store	Modern UI with cached queries and simple global state
Bundler	esbuild 0.28 (committed output in `public/app/`)	Fast, reproducible builds; users skip `npm install`
Charting	Custom canvas engine (`chart.js`, `indicators.js`, `strategy.js`)	Full control, offline, no licence constraints
Agent	TypeScript; `@anthropic-ai/sdk`, `@modelcontextprotocol/sdk`, `zod` 4	Model-agnostic adapters, MCP server, validated contracts
Agent storage	SQLite via `better-sqlite3` (WAL)	Durable, transactional, single-file
Documents	`unpdf`, `mammoth`, `@mozilla/readability`, `linkedom`	Read PDFs, Word and web pages safely
Models	DeepSeek Flash/Pro (default), Claude Haiku, Claude Code CLI, GPT-6 Luna/Sol, any OpenAI-compatible	Choice by cost, depth and vision
Data sources	Yahoo Finance, FRED, World Bank, Forex Factory, Nasdaq IPO calendar, SEC EDGAR, Tavily/Brave	Free/keyless where possible
Broker (paper)	Alpaca paper REST + `trade_updates` stream	Free fake-money US equities
Testing	`node:test`, Playwright 1.64, `@axe-core/playwright`	Unit, E2E, accessibility, visual
Secrets	macOS Keychain (`finsight-agent`), env fallback, 0600 files	Keys never in repo or prompts

12. Kestrel agent design

12.1 Principles

1. **Facts from code, words from the model.** Deterministic tools produce every figure; the model explains, compares and cites.
2. **Show the work.** Tool log, sources, asOf, cost and trace are part of every answer.
3. **Read-only toward the real world.** No tool can trade real money, move funds, message anyone or edit records; facts become proposals.
4. **Untrusted inputs stay data.** Web pages, filings and uploads are wrapped as untrusted and cannot issue instructions.
5. **Bounded.** Step, time, search, fetch and dollar limits per turn and per day.
6. **Model-agnostic.** Brains are plugins; no feature depends on one vendor.

12.2 Turn lifecycle

1. Validate input (Zod), check idempotency key, take the per-thread turn lock.
2. Build the **context pack**: a dated, compact snapshot of holdings, cash, policy and page context (symbol, timeframe, strategy).
3. Order the prompt for **prompt-cache** efficiency (stable system + tools first, volatile context last).
4. Reserve cost against turn and daily caps; call the brain with tool schemas.
5. Execute requested tools with schema validation; shape results with retrieval timestamps; wrap untrusted content.
6. Loop until the model answers or a limit is hit; stream events; persist message, trace, usage; reconcile cost.

12.3 Autonomy levels

Level	Meaning	Status
L0	Answer in the foreground	Shipped
L1	Background task (durable, budgeted, resumable)	Specified (FR-5)
L2	Scheduled or event-triggered runs	Specified, not built — prerequisite of FEAT-013
L3	Writes proposals the user confirms	Shipped for records; paper orders use the ticket gate

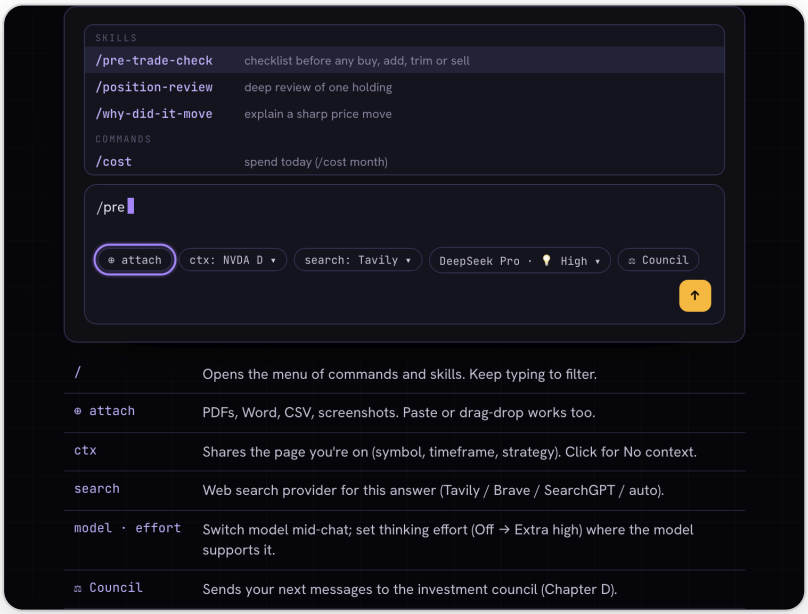
12.4 Limits and cost

Limit	Default	Range
Steps per turn	25	1–50
Seconds per turn	300	5–900

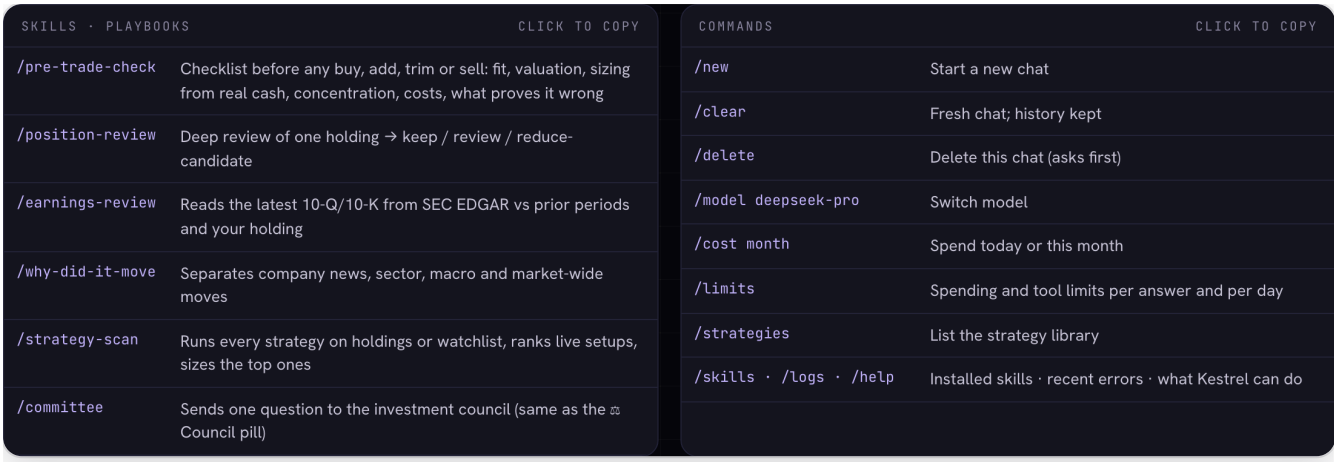
Limit	Default	Range
US$ per turn	0.25	> 0 – 100
US$ per day	5	> 0 – 1000
Searches per turn	20	configurable
Fetches per turn	25	configurable
Council research calls	8	configurable; optional `councilTurnUsd`, `councilOutputTokens`

12.5 Skills (playbooks)

Skill	What it does
/pre-trade-check	Objective fit, valuation, sizing from real cash, concentration, liquidity, costs/taxes, invalidation
/position-review	Why owned, weight and P/L, fundamentals trend, strategies, risks → keep / review / reduce-candidate
/earnings-review	Latest 10-Q/10-K from EDGAR vs prior periods and the holding; what the report changes
/why-did-it-move	Separates company, sector/peer, macro and market-wide drivers; review-trigger check
/strategy-scan	Runs every mechanical strategy on holdings/watchlist, ranks setups, sizes the top ones with event checks



GUIDE RECONSTRUCTION Screen 16 — The composer, control by control: attachments, context, search mode, model and thinking, council and lens



GUIDE RECONSTRUCTION Screen 17 — Commands and skills: `/` opens playbooks such as `/pre-trade-check` and `/why-did-it-move`

12.6 Observability

Every model call, tool call and data request is logged with timing and cost to `agent.db` and redacted JSONL logs (14-day retention). The UI exposes a per-answer trace; `/logs` shows recent errors; `/cost` reports spend by day or month.

13. Investment council design

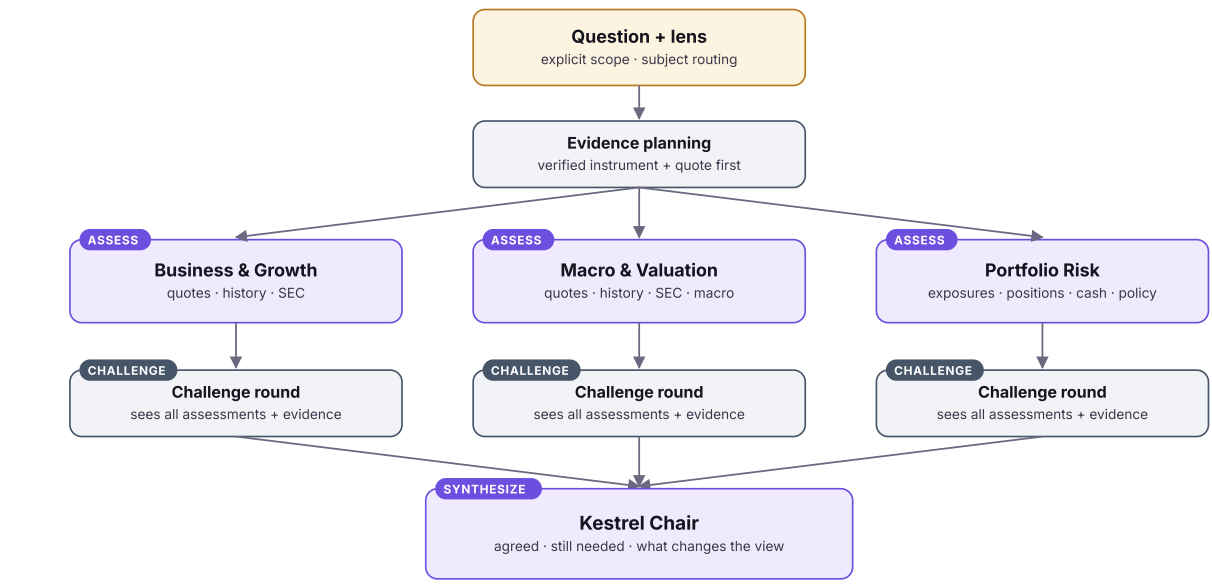


Figure 10 — Council pipeline

Aspect	Design
Roles	Business & Growth · Macro & Valuation · Portfolio Risk · separate Kestrel Chair
Stages	01 Assess (independent histories) → 02 Challenge (sees all assessments and evidence) → 03 Synthesize
Tools per role	Growth/Macro: quotes, histories, SEC filings/financials · Risk: exposures, positions, cash, policy · paid search excluded
Bounds	API brains only; concurrency 2; ≤ 4 tool requests + 1 follow-up per initial analyst; one validation repair per stage; deadline and context bound
Accounting	Conservative reservation before every call in turn and daily budgets; unknown usage keeps the reservation
Exclusivity	No other reply admitted while a council runs; a council is rejected while another turn runs
Cancellation	Aborts the whole tree; late results cannot complete a run; reload labels orphaned children interrupted
Contract	Versioned lenses and assessment schema in `council-contract.ts`; shape and reference integrity validated (not factual truth)
Qualified config	DeepSeek Flash, thinking off; 6/6 synthetic reliability rerun; no investment-performance advantage claimed

Aspect	Design
UI	Cards (verdict, evidence/against, uncertainty, would-change-if, chair row) and Chain of thought lanes; ⌘. switches; compact view in side panel; capsule status

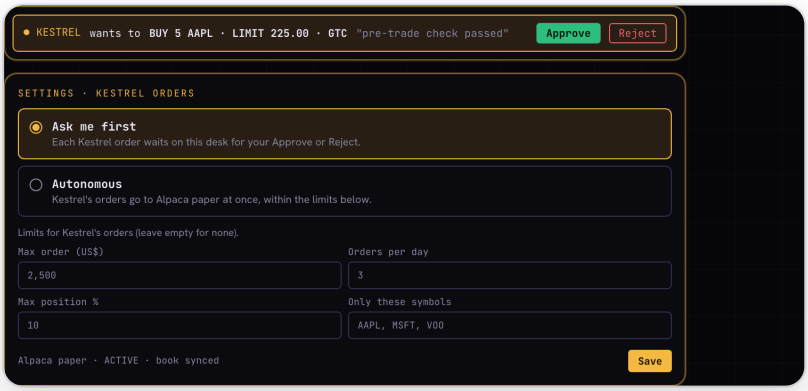


GUIDE RECONSTRUCTION Screen 18 — Chain of thought view (⌘.): what each specialist read, claimed and challenged, and how the chair weighed it

> **Known limitation**
>
> Research readiness stays **partial** while qualitative claim support lacks an independent check. Work item COUNCILQUALITY-001 tracks deeper evidence collection and honest readiness states.

14. Paper trading design

Concern	Design decision
Broker	Alpaca **paper** API only; base URL is a constant; no live-trading URL exists in code; tests point at a loopback fake (`FINSIGHT_FAKE_MARKET=1`)
Source of truth	Alpaca holds orders, fills, cash and positions; FinSight replays fill activities into `paper/portfolio.json` so every view works unchanged
Live updates	Server keeps `trade_updates` open while keys exist, resyncs on each fill and pushes events over `/api/paper/stream`
Valuation	Prices stay on the existing quote path (Yahoo, 15 s); Alpaca IEX data is not used for valuation
Orders	Market/limit · buy/sell · day/gtc · fractional shares · notional USD for market orders · cancel
Agent access	`paper_account` , `paper_place_order` , `paper_cancel_order` ; every order passes the server gate
Approval	`ask` (default) → pending ticket on the desk; `auto` → submit at once
Limits	Max order US$ · Kestrel orders per day · max position % after order · symbol allowlist; each off unless set; refusal reason returned
Settings authority	Changed only from the terminal (PIN session); the agent can read but not change them
Audit	Every order records origin (`user` / `kestrel` + thread) and appears in Orders, History and the Kestrel tab
Out of scope	Real-money trading, shorts/options in the book, Alpaca market-data streaming, scheduled agent runs, multi-account paper



`GUIDE RECONSTRUCTION` Screen 19 — Kestrel's paper orders: in Ask mode a pending ticket waits for Approve or Reject; the settings tab sets the approval mode and the limits

14.1 Roadmap: agentic trading (FEAT-013, planned)

A daily cycle: trigger → universe → screen (deterministic) → council research → structured `TradeProposal` (size from `size_position` , entry/stop/target, thesis, invalidation) → deterministic gate with new limits (stop required, max loss per trade, max open positions, daily drawdown kill switch, market

hours, earnings blackout) → bracket order with idempotency key → manage and score. The Alpaca MCP server may be used for **data only**; its trading toolsets are deliberately not given to Kestrel.

15. Security

15.1 Assets and trust boundaries

Assets: portfolio and financial data, broker paper keys, model API keys, the PIN/session, the agent token and proxy secret, and the integrity of answers.

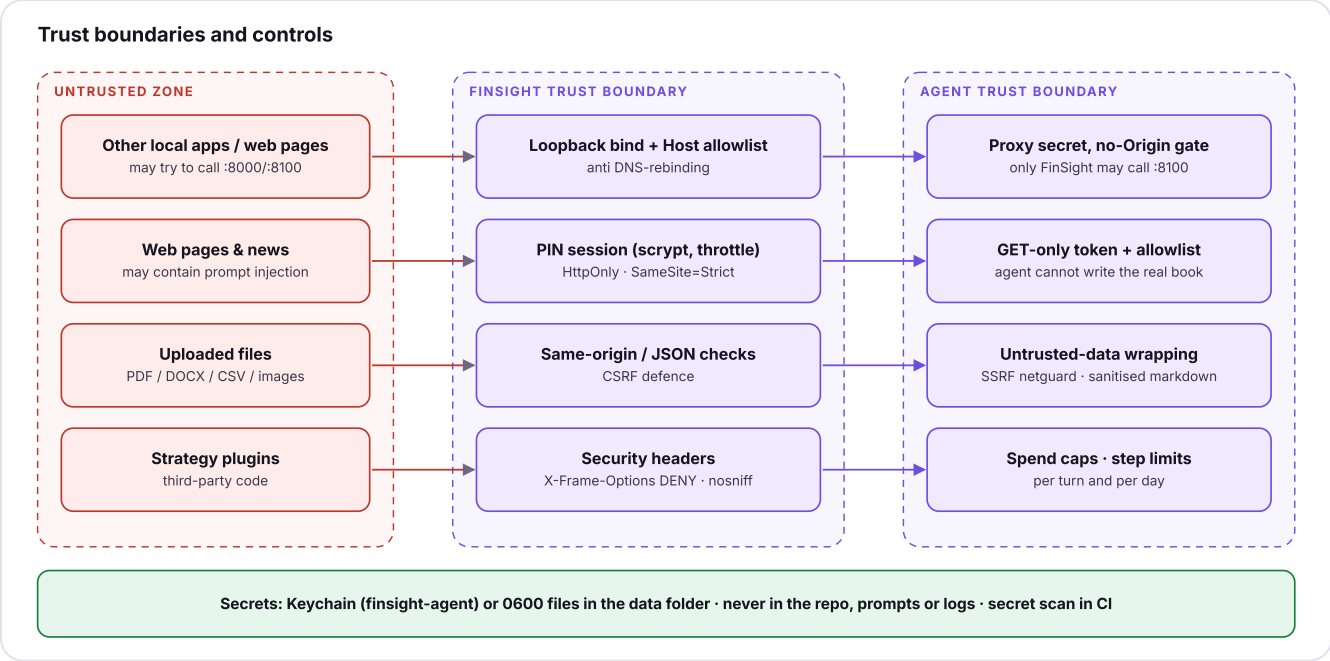


Figure 11 — Trust boundaries and controls

15.2 Threat model (STRIDE summary)

Threat	Example	Control	Verification
Spoofing	Another local process or web page calls :8000/:8100	Loopback bind, Host allowlist (anti DNS rebinding), PIN session, proxy secret, no CORS	Request without secret / wrong Host → 403
Tampering	CSRF changes the book	SameSite=Strict, same-origin (`Sec-Fetch-Site`) and Origin checks, JSON content type	Cross-origin POST rejected
Tampering	Agent edits the real book	GET-only token with endpoint allowlist; FinSight sole writer	Token POST → 403
Repudiation	Unclear who placed a paper order	Origin tag + thread on every order; traces and logs	Orders list shows origin
Info disclosure	Keys leak into logs/prompts/repo	Keychain/0600 files, redaction, secret scan in `agent:test` and CI, nothing personal in repo	Seeded secrets absent from logs/DB
Info disclosure	Screen sharing reveals wealth	UNPEEK blanks money figures everywhere	Visual check

Threat	Example	Control	Verification
Denial of service / cost	Runaway model loop	Step, time, search, fetch and dollar caps; daily cap	Loop test stops at limit
Elevation	Prompt injection in a web page or PDF	Untrusted wrapping, no write/send tools, proposals need confirmation	Injection eval set
Elevation	SSRF via `fetch_url`	Netguard blocks private, loopback, link-local, metadata and redirect-to-private	127.0.0.1, 10/8, 169.254.169.254 blocked
Elevation	Malicious strategy plugin	Child-process sandbox, permission manifest, timeout and memory cap	fs/net/spawn denied
Elevation	Paper agent over-trades	Approval mode, deterministic limits, settings PIN-only, paper URL constant	Limit tests against fake broker
XSS	Model output contains script	Sanitised markdown rendering	Script payloads neutralised
Supply chain	Compromised dependency	Zero runtime deps on server; pinned agent deps; lockfile; minimal surface	CI audit

15.3 Security controls checklist

- Both services bind to `127.0.0.1`; no `0.0.0.0`.
- Security headers: `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, Host header check.
- PIN: scrypt KDF off the event loop, brute-force throttling, optional idle lock.
- Agent link: random 32-byte token and proxy secret rotated at each server start (`agent-link.json`, 0600).
- Data files written atomically (temp + rename) with 0600 permissions in a folder outside the repo.
- API keys in macOS Keychain service `finsight-agent` (env fallback); paper keys server-side only.
- Demo and tests use synthetic data, fake market and fake broker — no real data or network.

16. Concurrency and consistency

FinSight is single-user but highly concurrent: the browser polls, several tabs may be open, Kestrel reads while the user writes, councils run calls in parallel and broker events arrive asynchronously. The controls below keep state consistent.

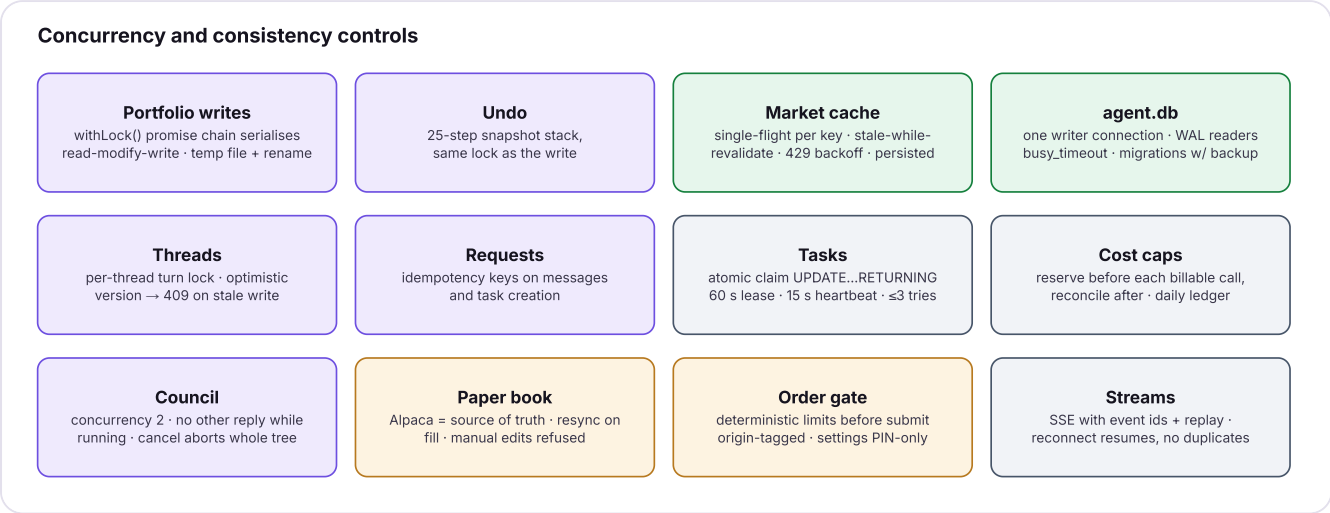


Figure 12 — Concurrency controls

Shared resource	Control	Failure it prevents
`portfolio.json`	`withLock()` promise chain serialises every read-modify-write; atomic temp-file rename	Lost updates from two quick trades or two tabs
Market cache	Single-flight per key; stale-while-revalidate; persisted last-good; 429 backoff	Thundering herd on Yahoo; blank panels during outages
`agent.db`	One writer connection; WAL for concurrent readers; `busy_timeout`	SQLITE_BUSY and torn writes
Chat threads	Per-thread turn lock; optimistic `version`, 409 on stale `expectedVersion`	Two turns interleaving in one thread
Requests	Idempotency keys on message and task creation	Double-submit on retry or flaky network
Background tasks	Atomic `UPDATE … RETURNING` claim; 60 s lease, 15 s heartbeat, ≤ 3 attempts	Two workers running one task; lost tasks after crash
Spend caps	Reserve before each billable call, reconcile after; persisted daily ledger	Parallel calls overshooting a cap
Council	Concurrency 2; exclusive with other turns; cancellation aborts the tree	Budget races, orphaned children
Plugin registry	Immutable snapshot swapped atomically on reload	Half-loaded plugins during a scan
Paper book	Alpaca as source of truth; resync on every fill; manual edits refused	Divergent local and broker state

Shared resource	Control	Failure it prevents
SSE streams	Event ids with replay on reconnect	Missed or duplicated events

17. Reliability and error handling

Situation	Behaviour
Yahoo / FRED outage or 429	Serve last-known-good from cache with stale flag; back off and retry in the background
Agent not installed / disabled / crashing	Panel shows the exact reason and fix (`npm run agent:install` , AGENT_AUTOSTART, log path); supervisor restarts with backoff
FinSight unreachable from agent	`FINSIGHT_DOWN` tool error; the model tells the user; nothing fabricated
Turn interrupted (crash, stop)	Partial message persisted and marked interrupted/stopped
Limit reached	Turn ends with a clear notice; `/limits` to adjust
Council subject ambiguous / data missing	Clarification or explicit gap; partial readiness
Paper key invalid or live	Refused before saving with a message
Paper limit breached	Order refused before reaching Alpaca with the reason
Paper stream drops	Status shown on the account strip; resync on reconnect
Corrupt or old data file	Migrations on load; undo snapshots; DB integrity check and backup before migrations

18. Software engineering practice and SDLC

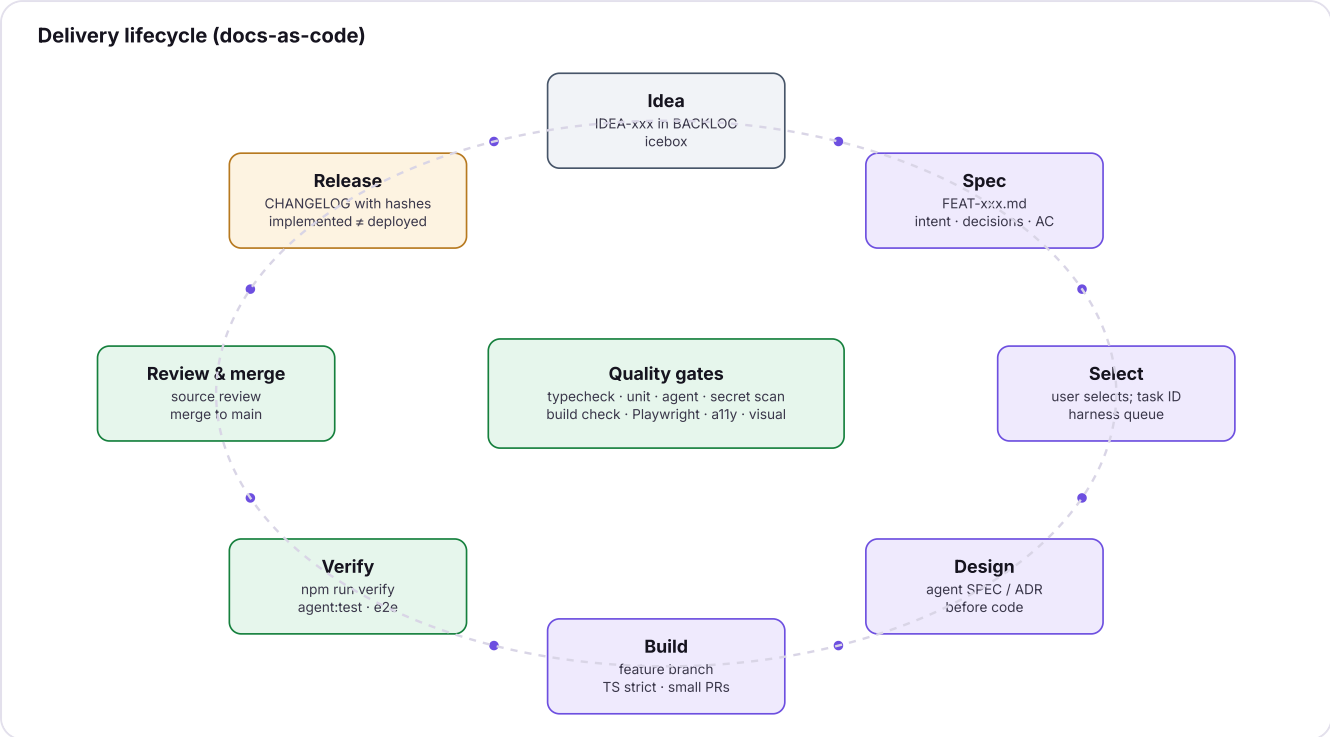


Figure 13 — Delivery lifecycle

18.1 Docs-as-code process

1. Ideas go to the **icebox** in `docs/BACKLOG.md` (IDEA-xxx); nothing is built from an idea.
2. Selected work gets a feature spec `docs/features/FEAT-xxx-<slug>.md` with intent, dated decisions, scope, out of scope and acceptance criteria.
3. Execution is scheduled in the investment-workspace harness (`harness/queue.json`, task IDs such as `PAPER-001`); user instructions override queue order.
4. Agent changes start from `agent/docs/SPEC.md`; contradictions need the user's approval; significant choices get an ADR.
5. Work happens on a feature branch, is verified, reviewed and merged to `main`.
6. `CHANGELOG.md` records integrated changes with commit/evidence hashes, distinguishing implemented, verified, merged and deployed.

18.2 Engineering standards

- TypeScript strict; erasable syntax only so Node runs `.ts` natively.
- Single-responsibility modules; `server.ts` routes only; pure math in its own modules (`returns.ts`, `portfolio.ts`).
- No LLM arithmetic anywhere in the product; deterministic tools own numbers.
- No personal data, keys or records in the repo; secret scan enforced.
- Accessible names match visible text; reduced-motion and keyboard paths for new UI.

- Commits are authored as the owner; small, reviewable changes; visual baselines refreshed deliberately.

18.3 Quality gates

Command	What it checks
`npm run typecheck`	Web, test and server TypeScript
`npm test`	Offline unit tests (server + web) with `node:test`
`npm run agent:test`	Agent typecheck, tests and secret scan
`npm run check:web`	Committed bundle matches the source build
`npm run e2e`	Playwright browser suite (fake market, fake Alpaca, fake agent)
`npm run verify`	All of the above — required before release

19. Testing strategy

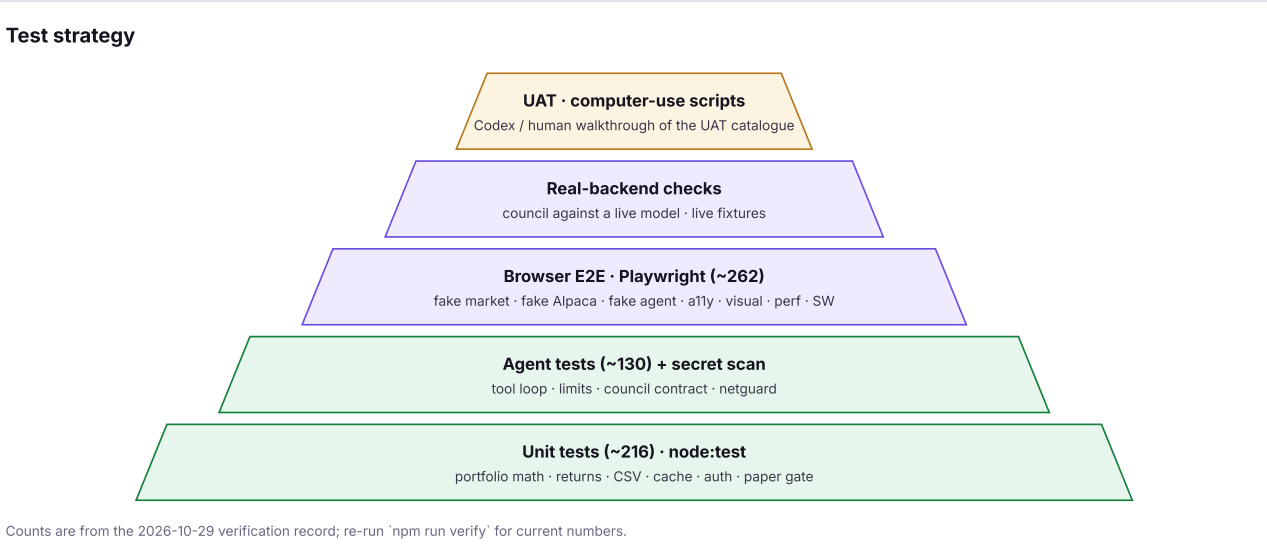


Figure 14 — Test pyramid

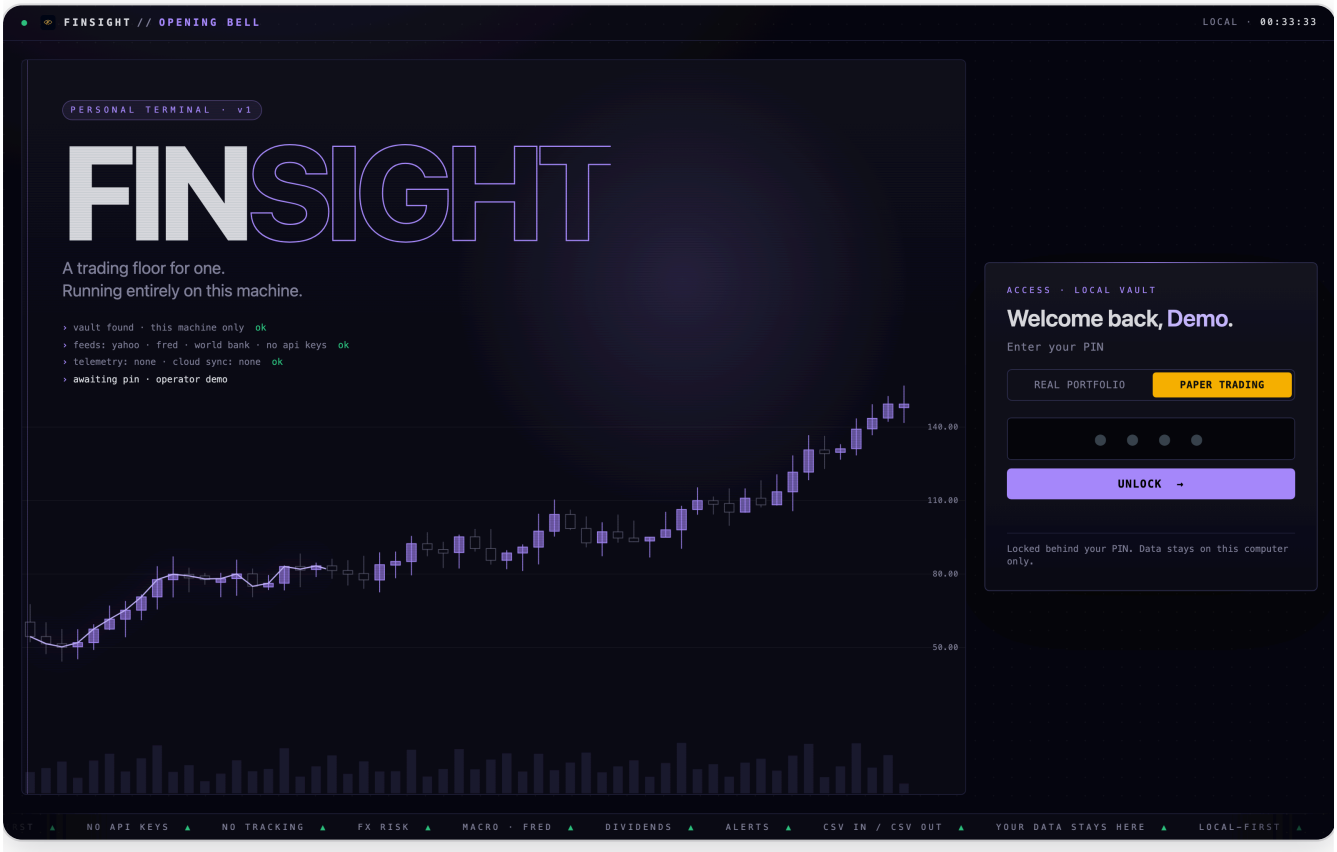
Level	Scope	Tooling / fixtures
Unit	Portfolio math, returns, FX split, CSV/IBKR parsing, cache, auth, paper gate and mirror	`node:test`, pure functions, fixtures in `test/fixtures`
Agent	Tool loop, limits, idempotency, council contract and orchestration, netguard, redaction	`node --test`, provider fakes, secret scan
Browser E2E	Every panel and flow, Kestrel streaming, council motion, paper desk, discover, layout drag	Playwright, `FINSIGHT_FAKE_MARKET=1`, `fake-agent.mjs`, fake Alpaca
Accessibility	Axe rules on terminal and Kestrel	`@axe-core/playwright`
Visual	Summary, positions, markets, detail at 1600/390 px, dark and light	Playwright snapshots
Performance & SW	Startup, perf budgets, service-worker upgrade and release	Dedicated specs
Real backend	Council against a live model on synthetic cases	Separate Playwright config
UAT	End-to-end acceptance by a human or Codex computer use	Section 20 catalogue

20. UAT plan for computer-use testing

20.1 Environment

1. Use a disposable data folder and the offline fakes: `FINSIGHT_DATA_DIR=<tmp> FINSIGHT_FAKE_MARKET=1 NOTIFY=0 PORT=8031 node server.ts` (or `npm run demo` for the seeded book, PIN 2468).
2. For paper tests with the fake broker, connect with test key id `PKE2ETEST1234567890` and the matching E2E secret from `web/e2e/PAPER.spec.ts` (fake broker accepts PK-prefixed ids).
3. For Kestrel tests without a model key, run `node web/e2e/fake-agent.mjs` and set `AGENT_URL` to it; for council tests use a configured DeepSeek key.
4. Viewport 1600×1000 for desktop cases, 390×844 for mobile cases. Record a screenshot at each Expected step.

> **For Codex computer use**
>
> Each case below is self-contained: preconditions, numbered steps using visible labels, and an observable expected result. Treat any mismatch in label text, numbers' presence or state as a failure and capture a screenshot.



LIVE CAPTURE Screen 20 — UAT starting point: the lock screen, where the tester picks REAL or PAPER and enters the demo PIN

20.2 UAT catalogue

ID	Title	Steps	Expected result
UAT-01	Unlock real book	Open app → REAL PORTFOLIO → type PIN → UNLOCK	Terminal loads; top bar shows REAL active; summary cards show S$ values
UAT-02	Wrong PIN	Type 0000 → UNLOCK	"Wrong PIN" shown; terminal stays locked
UAT-03	Operator card	Profile → set name, base SGD → Start empty → SAVE & CONTINUE	Dialog closes; greeting/profile uses name
UAT-04	Add via command	CMD: `ADD AAPL 10 195.50` → GO	AAPL row appears/updates; toast confirms; UNDO available
UAT-05	Undo	CMD: `UNDO` → GO	Previous change reverted; positions match prior state
UAT-06	Trade sheet	Positions → NVDA row → Buy → Shares 10 → save	Sheet shows You hold / Position after / New average before save; row updates after
UAT-07	Stock vs FX split	Read Unrealized, Stock P&L, FX P&L cards	Unrealized equals Stock + FX (within rounding)
UAT-08	Chart	Click VOO row → 1Y → Indicators → add RSI	Security Detail shows VOO; RSI pane appears
UAT-09	FX what-if	FX Risk → drag what-if slider to −5%	Impact on book shown negative; new value updates
UAT-10	Alert	CMD: `ALERT AAPL > 1` → GO	Alert listed; fires toast/notification (price above 1)
UAT-11	Discover tabs	Discover → US market → IPOs → Your symbols	Each tab renders with as-of; tab choice persists after reload
UAT-12	Layout	⚙ Layout → hide World Bank → DONE; drag Watchlist by ⠿	Panel hidden; order persists after reload
UAT-13	Light mode & UNPEEK	Click ◐ → click ◉ UNPEEK	Light theme applied; money figures blanked; click again restores
UAT-14	Open Kestrel	Press ⌘/	Panel opens with briefing tiles and suggestions; agent online
UAT-15	Kestrel answer	Ask "What is my currency and sector exposure?"	Tool log shows exposure tools; answer cites asOf; cost shown
UAT-16	Skill menu	Type `/` in composer	Commands and skills listed; `/pre-trade-check` selectable
UAT-17	Stop a turn	Ask a long question → press Esc	Turn stops; partial answer marked stopped
UAT-18	Delete chat	Type `/delete` → confirm	Chat removed from the Chats rail

ID	Title	Steps	Expected result
UAT-19	Council	⚖ Council → lens Balanced → "Should I trim NVDA before earnings?"	Stages Assess/Challenge/Synthesize progress; 3 cards + chair; cost shown
UAT-20	Council view switch	Press ⌘. during/after council	Toggles Cards ↔ Chain of thought
UAT-21	Switch to paper	Top bar → PAPER	Amber stripe/PAPER badge; connect panel if no keys; REAL data unchanged
UAT-22	Reject live keys	Enter key id not starting PK → Connect paper	Refused with message; nothing saved
UAT-23	Connect paper (fake)	Enter test PK key + secret → Connect paper	Desk appears; account strip shows equity; "Order stream live"
UAT-24	Market order	Ticket: BUY · SPY · Market · 1 share · Day → Buy SPY	Order fills; Positions (1); chart B marker; ledger entry
UAT-25	Limit + cancel	BUY · AAPL · Limit 1.00 · GTC → Buy; Open orders → Cancel	Dashed limit line appears then disappears; History shows cancelled
UAT-26	Kestrel ticket (ask)	Settings: Ask me first → ask Kestrel to buy 1 AAPL in paper	Pending ticket bar with Approve/Reject; Approve places order
UAT-27	Kestrel limit refusal	Settings: Max order US$ 10 → Save → ask Kestrel to buy 5 SPY	Order refused with reason; nothing sent
UAT-28	Manual edit refused in PAPER	In PAPER, CMD: `ADD AAPL 1 100`	Refused with explanation; orders are the only way in
UAT-29	Mobile layout	Viewport 390 px; scroll terminal and paper desk	No horizontal page scroll; ticket stacks below chart
UAT-30	Lock	Click 🔒	Returns to Opening Bell; API requests require PIN again

21. Deployment and operations

21.1 Run modes

Mode	Command	Port(s)	Data
Personal	`node server.ts` / autostart (`install-autostart.sh`)	8000 · agent 8100	`FINSIGHT_DATA_DIR` (e.g. ~/FinSightData)
Standalone (Mac)	double-click `start.command`	8002 · agent 8102	~/FinSightStandalone
Demo	`npm run demo` / `start-demo.command`	8003 · agent 8103	~/FinSightDemo (PIN 2468)
Test / UAT	`FINSIGHT_FAKE_MARKET=1 … node server.ts`	any	temporary folder

21.2 Configuration

Variable	Purpose
`PORT`, `HOST`	Listen port / host (loopback)
`FINSIGHT_DATA_DIR`	Data folder outside the repo
`AGENT_URL`, `AGENT_AUTOSTART`, `AGENT_DIR`, `AGENT_SHELL_ENV`	Agent location and supervision
`FINSIGHT_FAKE_MARKET`	Offline deterministic market + fake Alpaca for tests/demo
`FINSIGHT_CACHE_PERSIST`	Persist market cache to disk
`LOCK_IDLE_MIN`	Idle auto-lock minutes
`NOTIFY`, `ALERT_INTERVAL`	Desktop notifications and alert polling
`ALPACA_PAPER_KEY_ID`, `ALPACA_PAPER_SECRET_KEY`	Paper keys via environment (alternative to UI)
`DEEPSEEK_API_KEY`, `ANTHROPIC_API_KEY`, `TAVILY_API_KEY`	Model and search keys (prefer Keychain)
`KESTREL_UI`	Intelligent UI views setting

21.3 Runbook

Task	How
Install agent	`npm run agent:install`
Health	`node agent/src/cli.ts doctor`; `/api/agent-status`

Task	How
Logs	`agent.log` , `agent/logs/*.jsonl` ; `/logs` in Kestrel
Reset PIN	Delete `auth.json` in the data folder (holdings kept)
Reset demo	`npm run demo:reset`
Port clash	`PORT=9000 node server.ts`
Backup	Copy the data folder; `portfolio.json` + `undo.json` + `agent/agent.db`
Register MCP	`claude mcp add finsight -- node <finsight>/agent/src/cli.ts mcp`

22. Repository structure

```
finsight/
├── server.ts            Node HTTP server: routes, headers, session gate, proxies
├── lib/                 Domain modules (portfolio, market, macro, paper, auth, cache…)
├── web/                 React 19 + TS source
│   ├── src/features/    one folder per panel (positions, kestrel, paper, discover…)
│   ├── src/shell/       boot, layout, chart hooks
│   └── e2e/             Playwright specs, fake agent, fixtures
├── public/              served as-is: index.html, app/ bundle, chart.js, styles, sw.js
├── agent/               ↗ Kestrel (TypeScript)
│   ├── src/             api, chat, brains, tools, store, sandbox, mcp, obs
│   ├── skills/          bundled playbooks
│   ├── plugins/brains/  one manifest per model
│   └── docs/            SPEC, COUNCIL, INTELLIGENT-UI, ADRs
├── shared/              contracts shared by server/agent/web (council lenses)
├── scripts/             build-web, demo, checks
├── test/                node:test unit suites and fixtures
├── docs/                BACKLOG, features (FEAT-xxx), architecture, product PRD, guide,
master
└── AGENTS.md            rules for coding agents
```

23. Feature status and roadmap

Spec	Feature	Status (2026-11-01)
CORE-001	Kestrel research agent core	Integrated 2026-10-24
IUI-001 / START-001	Intelligent UI views; briefing start screen	Integrated 2026-10-26
FEAT-002	Investment lenses & council mode	Merged, verified, personally activated; quality work open (COUNCILQUALITY-001)
FEAT-003	Chat deletion, /delete, /clear, bulk clear	Merged — verified
FEAT-005	Prompt-cache optimisation	Merged — verified
FEAT-006 / 007 / 008	New-symbol trades; account transfers; IBKR fill import	Merged — verified
FEAT-009	Kestrel terminal redesign (panel states, light mode, council views)	Merged (PR #1)
FEAT-010	Market Discover: movers, US movers, IPO calendar	Merged locally; not deployed
FEAT-012	Paper trading, trade desk, Kestrel paper orders and limits	Merged to main (`aa373b7`); live run with real paper keys pending
FEAT-013	Agentic trading cycle	Plan, not selected
FEAT-001 / 011	IBKR MCP live sync; agent data sources	Deferred
IDEA-001…004	PDF memo export, ETF look-through, multi-broker, voice input	Icebox

24. Risks, assumptions and constraints

Type	Item	Mitigation / note
Risk	Unofficial Yahoo endpoint changes or rate-limits	Cache, backoff, stale flags; sources isolated in `market.ts`
Risk	Model hallucination or overconfidence	Tool-only numbers, citations, council gaps, figure flags, user decides
Risk	Prompt injection via web/docs	Untrusted wrapping; no write tools on the real book
Risk	Agent over-trading paper	Ask mode default, limits, PIN-only settings
Risk	Provider cost spikes	Turn and daily caps with reservation
Assumption	Single trusted user on a personal machine	Controls target other processes/pages, not other users
Assumption	FinSight running when the agent needs facts	Agent degrades with FINSIGHT_DOWN
Constraint	Forex Factory feed personal-use only; no redistribution	Local use only; disclaimer in README
Constraint	US-only market movers and IPOs	SG/HK deferred
Constraint	No ETF look-through yet	Exposure answers must say so

Disclaimer

FinSight is a personal tool and not investment advice. It is not affiliated with Bloomberg, Yahoo, Interactive Brokers, Alpaca, Forex Factory or the Federal Reserve. Data is provided as-is.

25. Engineering review against 24 domains

This section checks FinSight against a complete software-engineering checklist of **24 domains** in five groups. The checklist draws on the AWS Well-Architected pillars and OWASP ASVS. For each domain it states whether the domain applies to a **single-user, local** application, what the code does today (with file evidence), and the honest gaps. A domain marked *not applicable* is a design decision with its reason, not an omission.

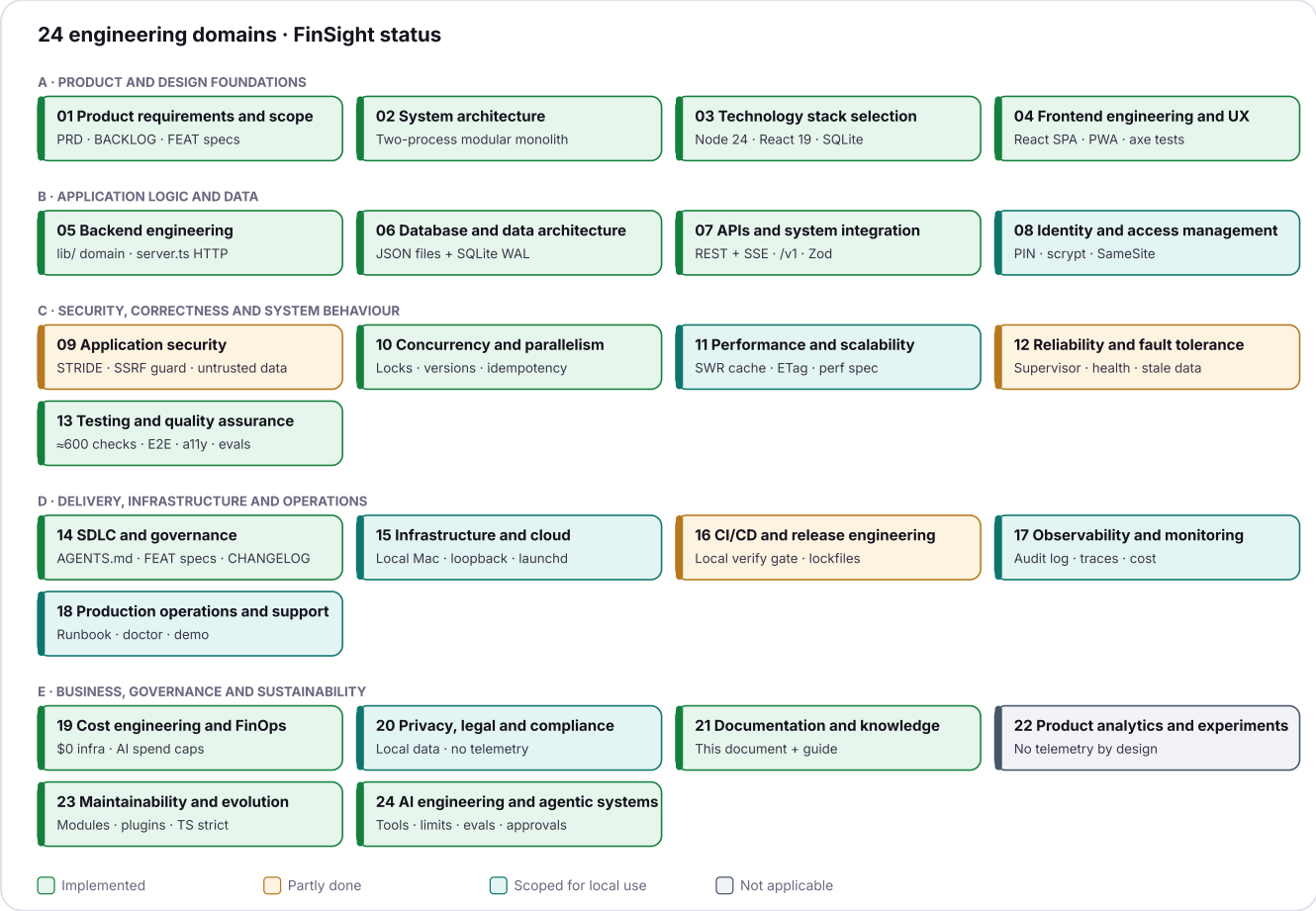


Figure 15 — The 24 engineering domains, coloured by FinSight's status

14	3	6	1
implemented	partly done	scoped for local use	not applicable

25.1 How the domains link together in FinSight

The domains are not separate features. They sit on the same layers. Product requirements drive the frontend, backend, data and integrations. Security, concurrency, testing and observability run across every layer, and the AI-agent domain spans all of them because Kestrel reads the book, calls tools and can place paper orders.



Figure 16 — Engineering layers with cross-cutting concerns, mapped to FinSight components

25.2 Domain-by-domain review

A. Product and design foundations

01 Product requirements and scope — IMPLEMENTED

HOW FINSIGHT ADDRESSES IT

- Problem, users and goals: Sections 1–4; `docs/product/PRD.md`.
- Functional requirements and user stories with acceptance criteria: Sections 5 and 7 (US-A1…US-F4, FR-*).
- Non-functional requirements: Section 8.
- Prioritisation and roadmap: `docs/BACKLOG.md` and one spec per feature in `docs/features/FEAT-001…013`, each with scope, non-goals and done criteria.
- Change control: a feature is specified before it is built, and `CHANGELOG.md` records what merged with Git references.

GAPS AND DECISIONS

- Success metrics are qualitative (G1–G6). There is no product telemetry by design, so adoption cannot be measured (see domain 22).

02 System architecture

IMPLEMENTED

HOW FINSIGHT ADDRESSES IT

- **Modular monolith in two processes**: the FinSight server (`server.ts` + `lib/`) and the Kestrel agent (`agent/`). They talk over loopback HTTP with a shared secret.
- Clear boundaries: domain maths in `lib/` (no I/O in the maths), HTTP in `server.ts`, UI in `web/`, agent tools behind one registry.
- Synchronous REST for commands; asynchronous SSE for streams (Kestrel events, paper `trade_updates`).
- Background work: alert checks, cache refresh, the Alpaca stream and agent turns run off the request path.
- Failure boundary: the agent can crash or be absent and the terminal keeps working. The server supervises and restarts it (domain 12).
- C4 diagrams in Section 9 (Figures 3–8).

GAPS AND DECISIONS

- Most decisions are recorded in feature specs and architecture notes. There is one formal ADR (`agent/docs/ADR-THINKING-EFFORT.md`), and a `docs/adr/` folder would make the rest easier to find.

03 Technology stack selection

IMPLEMENTED

HOW FINSIGHT ADDRESSES IT

- Node 24 runs TypeScript directly. The server has **zero runtime npm dependencies**, which keeps the supply chain small.
- Frontend: React 19, TanStack Query, Zustand, esbuild and a custom canvas chart. Agent: better-sqlite3, Zod, the Anthropic and MCP SDKs, Readability, unpdf and mammoth.
- All versions are pinned exactly and locked in `package-lock.json`. The license is MIT.
- Market data is public and keyless (Yahoo, FRED, World Bank, Nasdaq, SEC). Models are pluggable (DeepSeek, Claude, OpenAI-compatible, Claude Code CLI), which limits vendor lock-in.
- Full table in Section 11.

GAPS AND DECISIONS

- Yahoo endpoints are unofficial and can change without notice. This is mitigated by host fallback and the fake market for tests, but it remains a dependency risk (Section 24).

04 Frontend engineering and UX

IMPLEMENTED

HOW FINSIGHT ADDRESSES IT

- Component islands per panel (`web/src/features/*`), state in Zustand stores, server state in TanStack Query.
- Loading, empty, stale, error and offline states are explicit: stale quotes are flagged, and the agent shows an offline hint.
- Accessibility is tested with axe-core in Playwright (`H3.a11y`, `activity.a11y`). Panels can be reordered from the keyboard. Accessible names match the visible text.
- Responsive layout and an installable PWA with a service worker (release-tested in `release.sw.spec.ts`).
- Themes, fonts, accent palettes, light mode, and UNPEEK privacy mode for screen sharing.

GAPS AND DECISIONS

- SEO: not applicable. This is a private local app behind a PIN.
- i18n: English UI only. Currency and number formatting follow the base currency, and time zones are handled for exchanges and the agent.

B. Application logic and data

05 Backend engineering

IMPLEMENTED

HOW FINSIGHT ADDRESSES IT

- Business logic lives in `lib/` modules (portfolio, ledger, performance, dividends, paper, alerts). `server.ts` only routes, validates and serialises.
- Input validation at the HTTP boundary. The agent validates every tool input and every config file with Zod.
- Errors map to status codes: 400 for user errors, 502 for upstream failures, 409 for version conflicts in the agent.
- Configuration through environment variables and `config.json`, documented in Section 21.2.
- Background jobs: alert checker, cache refresh, the agent process supervisor and the paper stream with reconnect.
- Request lifecycle: Host check, then session check, then route, then a JSON response with `Cache-Control: no-store`.

GAPS AND DECISIONS

- There are no worker pools or queues. A single Node event loop is enough for one user, and CPU-heavy strategy runs go to a sandboxed child process.

06 Database and data architecture

IMPLEMENTED

HOW FINSIGHT ADDRESSES IT

- Two stores. The **portfolio** uses JSON documents in `FINSIGHT_DATA_DIR`, with 0600 permissions and atomic temp+rename writes. **Agent state** uses SQLite in WAL mode (`agent.db`).
- Schema evolution: `schema_migrations` is versioned, and the DB is **copied to `.bak-vN` before any migration** (`agent/src/store/db.ts`).
- Integrity: a corrupt `portfolio.json` is backed up to `.corrupt.json` instead of being overwritten (`lib/store.ts`).
- Transactions: SQLite transactions for multi-row agent writes, plus optimistic versions on threads.
- Retention: audit logs older than 14 days are pruned at start-up (`logRetentionDays`).
- Data model in Section 10.1 (Figure 9).

GAPS AND DECISIONS

- Multi-tenant isolation: not applicable (single user).
- Backups of the data folder are the user's job (Time Machine or a copy). There is no built-in scheduled backup or restore drill.

07 APIs and system integration

IMPLEMENTED

HOW FINSIGHT ADDRESSES IT

- FinSight `/api/` *is JSON over HTTP. The agent* `/v1/` is versioned, with Zod request schemas.
- SSE streams carry event ids and support **replay from Last-Event-ID**, so a dropped stream loses nothing.
- **Idempotency**: agent message posts accept idempotency keys. Paper ticket approval is claimed under a lock so it is sent once, and each order carries a unique `client_order_id` (`fs-u-…` for the user, `fs-k-…` for Kestrel).
- Timeouts on every outbound fetch (12 s, with a Yahoo host fallback), plus a stale-while-revalidate cache that serves the last good value during an outage.
- Kestrel's own rate limits: searches, page fetches, steps and seconds per turn (Section 12.4).
- MCP stdio server exposes the same tool registry to external coding agents.

GAPS AND DECISIONS

- No OpenAPI file. The API is documented in Section 10.2 and the agent's `SPEC.md` .
- No formal circuit breaker. Upstream failures degrade to cached or stale data instead.

08 Identity and access management

SCOPED DOWN

HOW FINSIGHT ADDRESSES IT

- One user and one credential: a PIN hashed with **scrypt** and compared with `timingSafeEqual` , stored 0600 (`lib/auth.ts`).
- **Per-IP brute-force throttle** on failed logins, so a third party cannot lock the owner out.
- Sessions: random token, `HttpOnly; SameSite=Strict` cookie, idle expiry (`LOCK_IDLE_MIN` , default 480 min), plus an hourly sweep of stale sessions.
- **Least privilege for the agent**: Kestrel's token can only place or cancel paper orders. It cannot change settings, limits, keys or the real book.
- Audit trail of agent actions (tool calls, orders with origin tags).

GAPS AND DECISIONS

- Registration, OAuth/OIDC, MFA, roles and tenants: not applicable. FinSight is single-user and local.
- PIN recovery is manual (delete `auth.json`), which is acceptable for a local owner.

C. Security, correctness and system behaviour

09 Application security

PARTLY DONE

HOW FINSIGHT ADDRESSES IT

- Threat model: STRIDE in Section 15.2, with trust boundaries in Figure 11.
- Loopback bind plus a **Host-header check against DNS rebinding**. Headers: `X-Frame-Options: DENY` and `X-Content-Type-Options: nosniff`.
- CSRF is closed by `SameSite=Strict` cookies on a loopback origin.
- **SSRF guard** in `fetch_url`: private and local addresses are blocked, DNS is pinned to the socket, and redirects, types and sizes are checked (`agent/src/tools/web.ts`).
- **Prompt-injection posture**: uploads and fetched pages are labelled untrusted data in the system prompt and are never treated as instructions.
- Secrets: model and broker keys are read from macOS Keychain first, and are never stored in the repo. `agent/scripts/secret-scan.ts` runs in `npm run agent:test`.
- Strategy code runs in a separate sandboxed child process (10 s and 256 MB limits).

GAPS AND DECISIONS

- **No Content-Security-Policy header yet.** Adding a strict CSP is a cheap defence-in-depth win.
- No automated dependency vulnerability scan (for example `npm audit` in the gate). The exposure is reduced by zero server runtime dependencies and exact pins.
- No external penetration test. This is reasonable for a personal tool, but it should be noted.

10 Concurrency and parallelism

IMPLEMENTED

HOW FINSIGHT ADDRESSES IT

- Server: `withLock` promise chains serialise writers to each JSON store. Writes are atomic (temp+rename). The cache is single-flight, so concurrent misses make one upstream call.
- Agent: SQLite single writer in WAL mode, a **per-thread lock with optimistic version (409 on conflict)**, idempotency keys, task leases, and **cost reservation before billable calls**, so parallel council calls cannot overspend.
- Council concurrency is capped at 2 specialist calls at a time. Cancellation aborts the whole call tree.
- Paper: Kestrel orders are serialised through a queue so they cannot race past the limits. Ticket approval is claimed under the orders lock. The book is rebuilt by replaying fills, so a duplicate event cannot double-count.
- Detail in Section 16 (Figure 12) and the worked example in 25.3.

GAPS AND DECISIONS

- None open.

11 Performance and scalability

SCOPED DOWN

HOW FINSIGHT ADDRESSES IT

- Stale-while-revalidate cache with a persisted copy, so start-up and panel loads are served from cache.
- Static assets are hashed by esbuild and served with an ETag. API responses are `no-store`.
- Performance checks: `web/e2e/perf.spec.ts`. The canvas chart draws thousands of candles without a DOM per bar.
- Prompt-cache optimisation for agent cost and latency (FEAT-005, `agent/scripts/cache-probe.ts`).

GAPS AND DECISIONS

- Horizontal scaling, load balancing, CDN and autoscaling: not applicable (one user, one machine).
- No load, soak or spike tests. Not needed at this scale.

12 Reliability and fault tolerance

PARTLY DONE

HOW FINSIGHT ADDRESSES IT

- Health endpoints: `/api/health` and `/v1/health`. `kestrel doctor` checks the agent setup.
- **Supervisor**: the server starts the agent, restarts it with backoff, and stops retrying after repeated crashes in 10 minutes, with a clear message (`lib/agentproc.ts`).
- Graceful degradation: when market data fails, the last good values are shown and flagged stale. When the agent is down, the terminal still works.
- Durability: atomic writes, corruption backup, a DB backup before migrations, and UNDO for book edits.
- launchd autostart keeps the server running and restarts it if it crashes (`install-autostart.sh`).

GAPS AND DECISIONS

- No formal SLO/RTO/RPO. The informal target is "the terminal is up when the Mac is on" and "no data loss beyond the last write".
- No automated backup restore test.

13 Testing and quality assurance

IMPLEMENTED

HOW FINSIGHT ADDRESSES IT

- Unit and integration: 29 root test files (`test/`), 24 agent test files, and web unit tests, all on Node's built-in test runner.
- E2E: 48 Playwright specs with about 240 tests against an isolated server, the **fake market** and a fake agent. A separate real-agent council suite.
- Accessibility (axe), visual baselines, service-worker release checks, and performance checks.
- Agent evaluation: council cases (`agent/evals/council-cases.ts`, `evaluate-council.ts`), UI qualification and a secret scan.
- UAT: 30 scripted cases for human or computer-use testers (Section 20).
- Release gate: `npm run verify` runs typecheck, unit tests, the web build check and E2E.

GAPS AND DECISIONS

- No coverage or mutation-testing report is published.

D. Delivery, infrastructure and operations

14 SDLC and governance — IMPLEMENTED

HOW FINSIGHT ADDRESSES IT

- `AGENTS.md` defines conventions for human and AI contributors: no personal data in the repo, no LLM arithmetic, spec-first changes.
- Backlog and feature specs, feature branches merged into `main`, and a `CHANGELOG` with real commit references.
- Definition of done: the feature's acceptance criteria are met, `verify` is green, and docs are updated (Section 18.3).
- Technical-debt tools: `scripts/coupling-inventory.mjs`, `scripts/css-dead.mjs`, and the legacy-maths parity tests.

GAPS AND DECISIONS

- Reviews are by the owner and AI reviewers. There is no second human reviewer.

15 Infrastructure and cloud — SCOPED DOWN

HOW FINSIGHT ADDRESSES IT

- Runs on the owner's computer, bound to `127.0.0.1`. `HOST=0.0.0.0` is an explicit opt-in for LAN access, and the Host check is relaxed only then.
- Environment isolation: separate data folders for real use, demo (`~/FinSightDemo`, port 8003) and tests (a temporary folder per run).
- Outbound only: market APIs, model providers and Alpaca paper. No inbound ports are exposed.

GAPS AND DECISIONS

- Cloud, containers, IaC, VPCs, gateways and data residency: not applicable. Nothing is hosted.

16 CI/CD and release engineering — PARTLY DONE

HOW FINSIGHT ADDRESSES IT

- Reproducible builds: exact versions and lockfiles, plus `npm run check:web` to confirm the built bundle matches the source.
- Release: merge to `main`, build, then restart the server (launchd). The agent rebuilds nothing at runtime.
- Rollback: `git revert` plus the DB `.bak-vN` copy made before a migration. Feature flags such as `KESTREL_COUNCIL` and approval modes allow progressive enablement.
- Smoke checks after start: health endpoints, `kestrel doctor`, and `startup.spec.ts`.

GAPS AND DECISIONS

- **No hosted CI.** The gate runs locally, so a GitHub Actions workflow that runs `npm run verify` on every push is the main gap.
- No release tags or versioned artifacts. `CHANGELOG` references commits instead.

17 Observability and monitoring

SCOPED DOWN

HOW FINSIGHT ADDRESSES IT

- Kestrel writes a structured JSONL **audit log** with 14-day retention, and keeps a **trace per turn** (model calls, tool calls, timings, tokens, cost) that is visible in the UI.
- Cost per answer and per day appears in the chat list. `kestrel logs` shows recent activity.
- The server shows live status (LIVE or stale) and the agent supervisor state with its last exit.

GAPS AND DECISIONS

- No metrics dashboard, distributed tracing or alerting. That fits one local user, and the price alerts are product features, not ops alerts.

18 Production operations and support

SCOPED DOWN

HOW FINSIGHT ADDRESSES IT

- Runbook and troubleshooting: Section 21.3 and the guide's "When something won't start" scene.
- Patch management: dependencies are pinned and upgraded deliberately. Node 24+ is enforced by the launcher and the autostart script.
- Break-glass: delete `auth.json` to reset the PIN, and `npm run demo:reset` rebuilds the demo.

GAPS AND DECISIONS

- On-call, escalation, SLAs and maintenance windows: not applicable. The owner is the operator.

E. Business, governance and sustainability

19 Cost engineering and FinOps

IMPLEMENTED

HOW FINSIGHT ADDRESSES IT

- Infrastructure cost is zero (local) and market data is free (keyless).
- **AI cost is the only variable cost.** Defaults are US$0.25 per reply and US$5 per day, and both can be edited (`/limits`).
- Costs are **reserved before** each billable call and settled afterwards, so parallel calls cannot overshoot the cap.
- Cost is shown per answer and per day. A cost tier per model guides model choice, and prompt caching lowers repeat cost.

GAPS AND DECISIONS

- None open.

20 Privacy, legal and compliance

SCOPED DOWN

HOW FINSIGHT ADDRESSES IT

- Data minimisation: holdings stay in a local 0600 folder outside the repo. There is no account and no telemetry.
- The data that leaves the machine is listed: questions and the tool results needed for them go to the chosen model provider, paper orders go to Alpaca, and symbol lookups go to public market APIs.
- `AGENTS.md` forbids committing personal data, keys or broker exports (`*.csv` is git-ignored).
- Licensing: the project is MIT, and dependencies are permissive.
- Not investment advice: the product and the council state this in the UI.

GAPS AND DECISIONS

- Singapore PDPA mainly governs organisations. An individual managing their own data in a personal capacity is outside its scope, and that would change if FinSight were offered to other users.
- Data-source terms of use (especially Yahoo) should be re-checked before any public distribution.

21 Documentation and knowledge

IMPLEMENTED

HOW FINSIGHT ADDRESSES IT

- This master document, the interactive guide (`docs/guide/finsight-guide.html`), `README` , `START-HERE` , `DOCS.md` , the agent `SPEC.md` and `COUNCIL.md` , the architecture notes and feature specs.
- Diagrams in C4 style (Section 9), a data model, sequences, and a UAT catalogue.
- Docs live with the code and change in the same commit as the code they describe (docs-as-code).

GAPS AND DECISIONS

- Only one formal ADR and no OpenAPI file (see domains 2 and 7).

22 Product analytics and experiments

NOT APPLICABLE

HOW FINSIGHT ADDRESSES IT

- Intentionally none. Privacy is a product goal (G3).
- Quality is measured offline instead: council evaluation metrics (`council-metrics.ts`), cost and latency traces, and UAT results.

GAPS AND DECISIONS

- If FinSight ever has other users, add opt-in, privacy-preserving analytics.

23 Maintainability and evolution

IMPLEMENTED

HOW FINSIGHT ADDRESSES IT

- Separation of concerns: pure maths in `lib/` with parity tests, islands per feature, and one tool registry.
- **Extensibility through manifests**: brains, tools and skills are plugins (`agent/plugins` , `SKILL.md`), so a new model or playbook does not touch the core.
- TypeScript typecheck across server, web and agent. A frontend migration is tracked in `docs/frontend-migration` .
- Data portability: CSV export of holdings, the journal and the ledger, plus plain JSON data files.

GAPS AND DECISIONS

- Solo ownership is a bus-factor risk. This document and `AGENTS.md` exist to reduce it.

24 AI engineering and agentic systems — IMPLEMENTED

HOW FINSIGHT ADDRESSES IT

- Model selection: pluggable brains with a cost tier and capabilities, and a default chosen by evaluation.
- Context engineering: the system prompt, the user's records and rules, the terminal context (symbol and range in view) and skills.
- **Tool calling over generation**: numbers come from deterministic tools, never from model arithmetic (an `AGENTS.md` rule). Answers carry an as-of time and sources.
- Structured output: the council contract is Zod-validated and figures in an assessment must cite evidence ids.
- Defences: untrusted-content labelling, an SSRF guard, read-only council tools, and an agent token that can do paper only.
- **Human approval**: record changes are proposals until the user confirms. Paper orders follow Ask mode (approve or reject) or Autonomous mode inside deterministic limits.
- Budgets: steps, seconds, searches, fetches, council calls, and dollars per turn and per day.
- Evaluation and observability: council evals, per-turn traces and cost.
- Fallback: switching models is one menu choice, and when the agent is down the terminal keeps working.

GAPS AND DECISIONS

- Evaluation sets are small and synthetic. They show that the pipeline works, not that the investment calls are good (stated in `CHANGELOG`).
- No RAG index over the user's documents. Uploads are read per turn instead.

25.3 Worked example: two orders for the same paper position at once

The checklist's "last item in stock" example has a direct FinSight equivalent. The user clicks **Sell 8 AAPL** on the desk while Kestrel, in Autonomous mode, sends **Sell 8 AAPL** for the same position. Every domain has a job in getting this right:

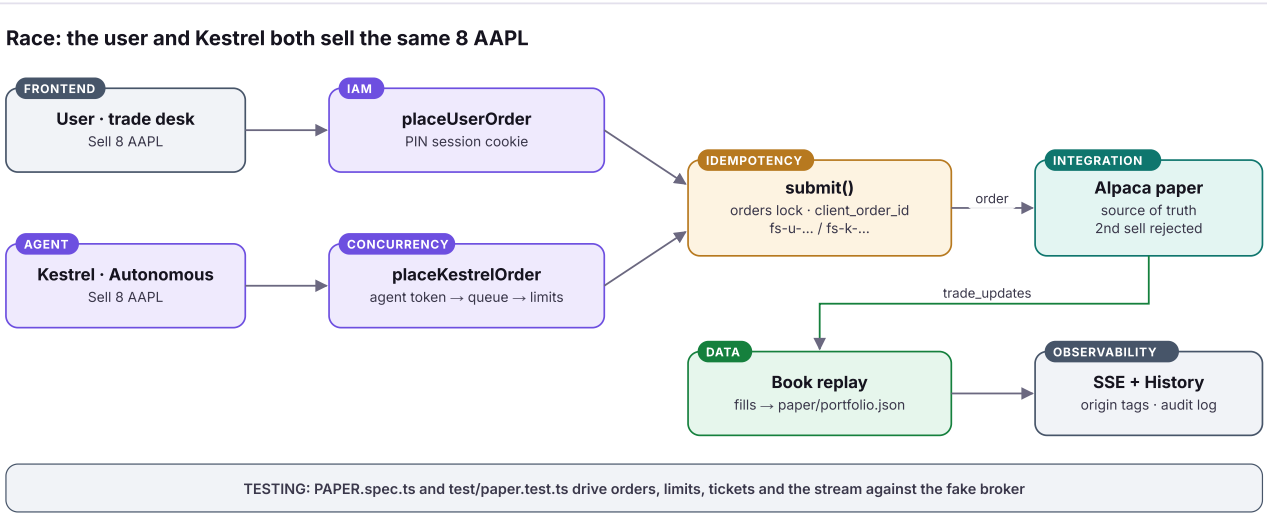


Figure 17 — One race, many domains

Domain	What happens in FinSight
Frontend	Both the desk and the Kestrel tab show 8 shares held. The ticket shows the estimate, not a promise.
Backend / API	Both requests reach `POST /api/paper/orders`. The user's request is routed to `placeUserOrder` and the agent-token request to `placeKestrelOrder`.
Identity	Only the agent token reaches the Kestrel path, and that path cannot change settings or limits.
Concurrency	Kestrel's orders run **one at a time** through a queue (`kestrelQueue` in `lib/paper.ts`), so two Kestrel calls cannot both pass the orders-per-day and position checks before either is recorded. Order metadata is written under the orders lock.
Idempotency	Approving a Kestrel ticket **claims it inside the orders lock** (pending → approving), so a double click or a second tab cannot send the same order twice. Every order carries a unique `client_order_id` (`fs-u-…` user, `fs-k-…` Kestrel) that ties each fill back to its origin.
Integration	Alpaca is the source of truth. The second sell is rejected there (insufficient quantity), and the rejection is shown to whoever sent it.
Data	The paper book is rebuilt by replaying fills from `trade_updates`, so it cannot record a sale that did not fill, and a duplicate event cannot double-count.
Reliability	If the stream drops after a fill, the reconnect plus `/api/paper/sync` reconciles the book with Alpaca.
Observability	Both orders keep their origin tag (user or Kestrel), so History shows who did what. The agent audit log has the turn that sent it.
Testing	`PAPER.spec.ts` and `test/paper.test.ts` cover orders, limits, tickets and stream updates against the fake broker.

25.4 Priorities: MVP, production, high scale

Not every domain needs enterprise depth. The matrix shows the expected level at each maturity stage and where FinSight stands. FinSight is a **production-grade personal tool**: it needs production-level correctness, security and testing, but not scale-out infrastructure.

Engineering domain	MVP	Production	High scale	FinSight today
Requirements and scope	Required	Required	Required	Done: PRD, stories, FEAT specs
Architecture and tech stack	Required	Required	Required	Done: two-process modular monolith
Frontend, backend, database	Required	Required	Required	Done
Authentication and authorisation	Required	Required	Required	Done for one user: PIN, sessions, agent token
Baseline security and data protection	Required	Required	Required	Mostly done; add a CSP and a dependency scan

Engineering domain	MVP	Production	High scale	FinSight today
Concurrency correctness	Required	Required	Required	Done: locks, versions, idempotency, reservations
Automated testing	Basic	Comprehensive	Comprehensive	Comprehensive: about 600 checks
CI/CD and deployment	Basic	Mature	Advanced	Basic: local gate, no hosted CI
Monitoring and alerting	Basic	Mature	Advanced	Basic: audit log and traces
Backups and recovery	Basic	Tested	Advanced	Basic: migration and corruption backups, manual folder backup
Performance optimisation	Targeted	Measured	Extensive	Targeted: caching, perf spec
Load balancing and autoscaling	Optional	As needed	Usually	Not needed
Distributed services	Usually unnecessary	As needed	As needed	Not needed: two local processes
Disaster recovery	Risk-based	Planned	Regularly tested	Risk-based: a local copy of the data folder
Cost management	Basic	Active	Advanced	Active for AI spend: caps and reservations

> **Backups start on day one**
>
> Backups matter as soon as valuable data is stored. FinSight backs up before migrations and on corruption, but the data folder itself should also be in the user's normal backup (Time Machine or an encrypted copy).

25.5 Engineering documents

These are the recommended documents and where each concern lives in FinSight. Several are combined into this master document, which is fine at this stage as long as each concern has one clear home.

Recommended document	Where it lives in FinSight	Status
`PRD.md`	`docs/product/PRD.md`; Sections 1–7	Present
`NFR.md`	Section 8 of this document	Present (in this document)
`ARCHITECTURE.md`	`docs/architecture/SYSTEM.md`, `finsight-architecture.md`, `kestrel-architecture.md`; Section 9	Present

Recommended document	Where it lives in FinSight	Status
`ADR/`	`agent/docs/ADR-THINKING-EFFORT.md`; other decisions inside feature specs	Partial: one ADR, no ADR folder
`TECH_STACK.md`	Section 11; `package.json` files	Present (in this document)
`DATA_MODEL.md`	Section 10.1; `agent/docs/SPEC.md`	Present (in this document)
`API_SPEC.yaml`	Section 10.2; `agent/docs/SPEC.md`	Gap: no machine-readable OpenAPI
`SECURITY.md`	Section 15; `agent/docs/SPEC.md` §10	Present (in this document)
`TEST_PLAN.md`	Sections 19–20; `web/e2e/COVERAGE.md`	Present
`DEPLOYMENT.md`	Section 21; `README.md`, `START-HERE.md`	Present
`OPERATIONS.md`	Section 21.3; the guide's troubleshooting scene	Present
`ROADMAP.md`	`docs/BACKLOG.md`; Section 23	Present
`AGENTS.md`	`AGENTS.md` (repo root)	Present

25.6 The five engineering gates

Coding agents can write features quickly, but speed does not prove a change is correct, secure or ready to run. FinSight uses five gates, and each one maps to concrete checks:

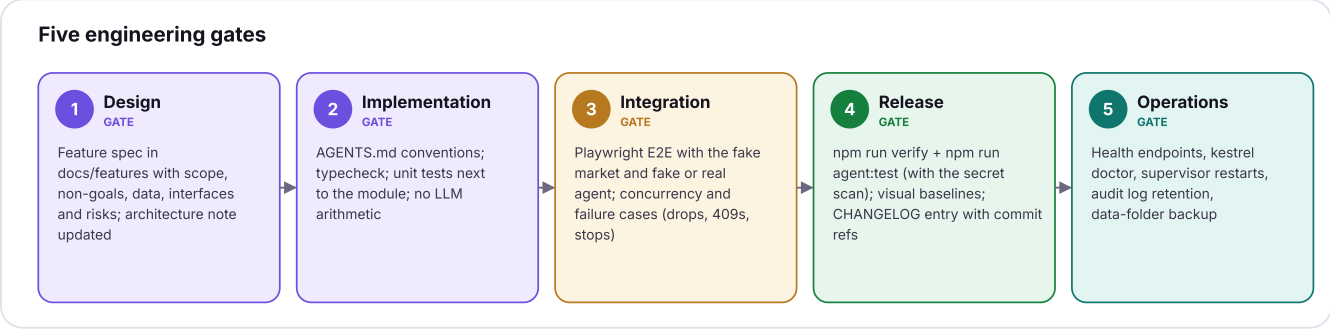


Figure 18 — Five engineering gates and the FinSight checks behind each

25.7 Improvement backlog from this review

Priority	Item	Domain	Why
P1	Add a hosted CI workflow that runs `npm run verify` and `npm run agent:test` on every push	16 CI/CD	The gate currently depends on someone remembering to run it
P1	Add a strict Content-Security-Policy header	09 Security	Defence in depth against injected script in rendered content

Priority	Item	Domain	Why
P2	Add a dependency audit (`npm audit --omit=dev` and the agent's dependencies) to the gate	09 Security	Agent and web dependencies are the main supply-chain surface
P2	Document and test a data-folder backup and restore	06 / 12	No restore drill exists
P3	Start `docs/adr/` with the key past decisions (local-only, zero-dependency server, JSON + SQLite, Alpaca paper only)	02 / 21	Makes the reasons findable for new contributors and agents
P3	Publish an OpenAPI description of `/api` and `/v1`	07 / 21	Contract tests and external tools

Appendix A — Glossary

Term	Meaning
Base currency	Currency all totals are reported in (default SGD)
Stock P&L / FX P&L	Part of unrealized gain from the price move / from the exchange-rate move
Blended entry	Weighted-average exchange rate paid for foreign holdings
TWR / XIRR	Time-weighted return (ignores cash-flow timing) / money-weighted personal return
VaR	Value at risk; here 1-week, 1σ and 95% from recent FX volatility
Lot	One purchase with quantity, price, FX and time
Brain	Model plugin used by Kestrel for a turn
Tool	Function the model can request; executed by deterministic code
Skill	Playbook in SKILL.md format invoked with `/name`
Context pack	Dated snapshot of the user's situation built by code for each turn
Lens	Analytical emphasis for the council (Balanced, Growth, Speculative, Hedge Fund, Macro)
Ticket	A Kestrel paper order waiting for Approve/Reject
UNPEEK	Privacy mode that blanks money figures
MCP	Model Context Protocol; exposes tools to external AI clients

Appendix B — Terminal commands

Command	Effect
`ADD SYM QTY PRICE [DATE]`	Buy/add (averages in); optional date fetches historical FX
`SELL SYM QTY [PRICE]`	Sell; realized P&L recorded
`DEL SYM`	Stop tracking
`CASH CCY AMOUNT`	Set cash
`UNDO`	Undo last change
`LEDGER`	Open trades ledger
`Q SYM` / `SYM`	Load symbol detail
`WATCH` / `UNWATCH SYM`	Watchlist
`ALERT SYM > | < PRICE`	Price alert

Command	Effect
`NEWS SYM`	News
`WHT MKT PCT`	Withholding rate
`MKT` · `HELP`	Refresh markets · help

Appendix C — Kestrel commands

Command	Effect
`/new` · `/clear` · `/delete`	New chat · fresh chat keeping history · delete (asks first)
`/model <id>`	Switch brain (e.g. `deepseek-pro`)
`/cost [month]`	Spend
`/limits`	Spending and tool limits
`/strategies` · `/skills` · `/logs` · `/help`	Library · installed skills · recent errors · capabilities
`/committee <question>`	Run the council once
`/pre-trade-check` · `/position-review` · `/earnings-review` · `/why-did-it-move` · `/strategy-scan`	Skills
⌘/ · ⌘⇧/ · ⌘. · Esc	Toggle panel · full page · council view · stop

Appendix D — Traceability matrix

Story	Functional requirements	Automated evidence	UAT
US-A1/A2	FR-ACC-1…4	lock, session, PAPER specs	UAT-01, 02, 21, 30
US-A3/A4	FR-ACC-5	startup, demo safety	UAT-03
US-B1/B2	FR-PF-1, FR-PF-2	portfolio unit tests, summary visual	UAT-07
US-B3/B4/B5	FR-PF-3…7	add-sell, import-export, trades, ledger, OPENFEATURES	UAT-04, 05, 06
US-B6/B7/B8	FR-AN-1…3	returns/performance units, panels spec	UAT-09
US-C1…C5	FR-MD-1/2, FR-DS-1, FR-MC-1, FR-AL-1	chart, DISCOVER, F-specs, visual	UAT-08, 10, 11, 12, 13

Story	Functional requirements	Automated evidence	UAT
US-D1…D8	FR-K-1…11	G.agent, KESTRELUX, CHATDELETE, LIMITS, IUI, agent tests	UAT-14…18
US-E1…E4	FR-C-1…6	COUNCIL.agent, COUNCIL.motion, council real-backend	UAT-19, 20
US-F1…F4	FR-P-1…7	PAPER.spec, paper unit tests vs fake broker	UAT-21…29

Appendix E — References

- `README.md`, `DOCS.md`, `AGENTS.md` — overview, developer/API reference, agent rules
- `docs/architecture/SYSTEM.md`, `finsight-architecture.md`, `kestrel-architecture.md`
- `agent/docs/SPEC.md` (SRS), `COUNCIL.md`, `INTELLIGENT-UI.md`, `ADR-THINKING-EFFORT.md`
- `docs/product/PRD.md`, `docs/BACKLOG.md`, `docs/features/FEAT-001…013`
- `docs/guide/finsight-guide.html` — interactive demo guide and user manual
- `web/e2e/COVERAGE.md` — browser test coverage map